



Факултет по Компютърни Системи и Технологии

ДИПЛОМНА РАБОТА

на тема

Проектиране и разработване на уеб система
за риболовни дневници и обогатяване на
информационната база на българския
риболов

Студент: Кристиян Георгиев Златковски

Факултетен номер: 121320043

Специалност: Компютърни системи и технологии

Образователно-квалификационна степен: Магистър

Научен ръководител: гл. ас. д-р инж. П. Данов

• София, 2021 г. •

Съдържание

Увод.....	5
1. Анализ на темата и съществуващи решения. Цели и задачи	6
1.1. Преглед на най-често използваните риболовни приложения	6
1.1.1. ANGLR.....	6
1.1.2. iAngler и iAngler Tournament	7
1.1.3. Fishinda.....	7
1.1.4. FishTrack - Fishing Charts	8
1.1.5. Ribolovenatlas	9
1.1.6. Ribarnik.com.....	9
1.1.7. Pro Angler Fishing App	10
1.2. Обобщение от направения преглед на риболовни приложения	11
1.3. Цели, задачи и характеристики на разработката	11
1.3.1. Основни цели	11
1.3.2. Основни задачи.....	12
Задача 1	12
Задача 2	12
Задача 3	12
1.3.3. Потребителски характеристики.....	12
2. Определяне на специфичните изисквания към програмния продукт.....	13
2.1. Потребителски изисквания.....	13
2.1.1. Нерегистриран потребител.....	13
2.1.2. Регистриран потребител.	13
2.1.3. Регистриран потребител с ограничени функции.	13
2.1.4. Регистриран потребител (Модератор).....	13
2.1.5. Регистриран потребител – управител (администратор).....	14
2.2. Изисквания към използвания модел.....	14
2.2.1. Част нерегистриран потребител.....	14
2.2.2. Част регистриран потребител.....	14
2.2.2.1. Въвеждане на данни.	14
2.2.2.2. Промяна на данни.	15
2.2.2.3. Преглед на данните.....	15

2.3.	Функционални изисквания	16
2.3.1.	Потребител Class 1 – Регистриран потребител.	16
2.3.1.1.	Функционално изискване 1.1.	16
2.3.1.2.	Функционално изискване 1.2.	16
2.3.1.3.	Функционално изискване 1.3	16
2.3.1.4.	Функционално изискване 1.4.	17
2.3.1.5.	Функционално изискване 1.5.	18
2.3.1.6.	Функционално изискване 1.6.	19
2.3.1.7.	Функционално изискване 1.7.	19
2.3.1.8.	Функционално изискване 1.8.	21
2.3.2.	Потребител Class 2 – Модератор.....	21
2.3.2.1.	Функционално изискване 2.1	21
2.3.2.2.	Функционално изискване 2.2	22
2.3.2.3.	Функционално изискване 2.3	22
2.3.2.4.	Функционално изискване 2.4	23
2.3.3.	Потребител Class 3 – Администратор.....	23
2.3.3.1.	Функционално изискване 3.1	23
2.3.3.2.	Функционално изискване 3.2	23
2.3.3.3.	Функционално изискване 3.3	24
2.3.3.4.	Функционално изискване 3.4	24
2.3.3.5.	Функционално изискване 3.5	25
2.4.	Нефункционални изисквания	26
2.4.1.	Изисквания за сигурност на системата	26
2.4.1.1.	Нефункционално изискване 1.1	26
2.4.1.2.	Нефункционално изискване 1.2	26
2.4.1.3.	Нефункционално изискване 1.3	26
2.4.1.4.	Нефункционално изискване 1.4	26
2.4.2.	Изисквания към потребителския интерфейс	27
2.4.2.1.	Нефункционално изискване 2.1	27
2.4.2.2.	Нефункционално изискване 2.2	27
2.4.3.	Изисквания към потребителския интерфейс	27
2.4.3.1.	Нефункционално изискване 3.1	27
2.4.3.2.	Нефункционално изискване 3.2	27
2.4.3.3.	Нефункционално изискване 3.3	27

2.4.3.4.	Нефункционално изискване 3.3	28
3.	Архитектура на продукта и модел на данните.....	29
3.1.	Избор на средства за реализация	29
3.1.1.	Програмен език.....	29
3.1.2.	Визуализация	29
3.1.3.	Модел на БД.....	29
3.1.4.	Архитектура	30
3.1.4.1.	Моделът (Model).....	31
3.1.4.2.	Изглед	31
3.1.4.3.	Контролер.....	32
3.1.5.	Модел на данните	32
3.1.6.	Детайлно описание на използваните таблици	33
3.1.4.1.	Таблица Стръв (Bait)	33
3.1.4.2.	Таблица Риби (Fish).....	33
3.1.4.3.	Таблица Коментари (Comments).	33
3.1.4.4.	Таблица Риболовни записи (Journal).	34
3.1.4.5.	Таблица Предложения за вид риба (fishProposals).	35
3.1.4.6.	Предложения за добавяне на риба към водоем (fishToWaterProposals).	35
3.1.4.7.	Потребителски роли (role).	35
3.1.4.8.	Турнири (tournament).....	36
3.1.4.9.	Награди от турнири (tournament_rewards).	36
3.1.4.10.	Потребители (user)	37
3.1.4.11.	Водоеми (water).....	37
3.1.4.12.	Цялостна архитектура на използваната БД.	37
4.	Функционалност на разработеното уеб приложението	39
4.1.	Програмна реализация	39
4.2.	Ръководство на потребителя	43
	Заклучение	57
	Използвани програмни продукти	58
	Използвана литература	58

Увод

Риболовът съществува от поне 40 000 години и през вековете е бил съществена част от живота на хората. Той бил ценен източник на храна и една от многото причини поради които хората започват да строят своите примитивни жилища в близост до водоеми. Умението да улавяш риба било жизненоважно и тези които били добри в това биват ценени от цялото племе.

Въпреки ,че от тогава са изминали хиляди години , днес множество хора все още се изхранват чрез риболов, някои директно като използват уловената риба за храна а други индиректно като продават или заменят уловената риба за да добият пари или други провизии. Градовете построени близо до големи водоеми като реки и морета са едни от най- проспериращите в света и голяма част от заслугата за това е именно на риболова.

Разбира се риболовът в наши дни вече не е само метод за прехрана. Много хора гледат на него като средство за релаксиране и откъсване от напрегнатото ежедневие. Риболовът носи усещане за успех и заслужена награда всеки път, когато след часове чакане най-накрая се постигне успех, ефектът се засилва дори повече ако уловената риба е с рекорден размер.

В сегашната разработка ще разгледаме уеб приложение за създаване на риболовни дневници и събиране на риболовни данни в единна система. Чрез него потребители из цялата страна ще могат да споделят своите риболовни успехи и наблюдения със своите колеги по хоби. Ще подпомогнат за обогатяване на информационната база за българския риболов и ще бъдат наградени за това чрез различните риболовни турнири.

В първа глава от разработката ще разгледаме подобни на разработваното, приложения. Ще оценим техните предимства и недостатъци за да може тази информация да послужи като отправна точка в създаването на нашето приложение. В тази глава също така ще си поставим цели и задачи, които искаме да бъдат постигнати в процеса на работа върху приложението.

Във втора глава от разработката ще разгледаме различните изисквания които уеб приложението трябва да покрие , потребителски , изисквания към модела и функционални изисквания.

В трета глава ще разгледаме избраната работна архитектура, използваните за постигането и технологии и описание на модела на работа.

В глава четири ще разгледаме по детайлно някои от функционалностите на приложението и ще направим пълен преглед на уеб приложението от гледна точка на потребителя.

1. Анализ на темата и съществуващи решения. Цели и задачи

1.1. Преглед на най-често използваните риболовни приложения

1.1.1. ANGLR.

Приложението предлага следните основни функционалности:

- ✓ възможност за изготвяне на риболовни дневници и правене на записи включващи (локация, снимки, дължина, тегло, бележки и стръв);
- ✓ възможност за изготвяне на риболовни маршрути с помощта на GPS;
- ✓ автоматично измерване и записване на влажността на въздуха и температура на околната среда и водата;
- ✓ предоставя статистически данни за рибите в региона, използвана стръв и предишни улови;
- ✓ възможност за споделяне на запазени маршрути и данни за предишни улови с приятели;
- ✓ бърз достъп до магазини предлагащи подходяща стръв за планирания вид риба.

Предимства:

- приложението е безплатно за изтегляне с заплащане за допълнителни функции.
- възможност за комуникиране с останалите потребители със система подобна на Facebook.

Недостатъци:

- ограничение на данните за водоемите до района на САЩ.
- няма възможност за работа офлайн (офлайн карта или дневник със записите).
- ограничено използване на приложението до Андроид и IOS за използване на лаптоп е необходимо използването на допълнителни емулятори.
- няма възможност за смяна на езика на интерфейса на български.

1.1.2. iAngler и iAngler Tournament

Приложението предлага следните основни функционалности:

- ✓ информация и класиране за различни риболовни турнири в региона чрез приложението iAngler Tournament.
- ✓ възможност за изготвяне на риболовни дневници и правене на записи включващи (снимки, вид на екземпляра и допълнителна информация).
- ✓ предоставя информация за различните видове риба (снимка на рибата, дълбочина на кълване, предишни улови).

Предимства:

- приложението е безплатно за изтегляне с заплащане за допълнителни функции;
- възможност за комуникиране с останалите потребители със система подобна на Facebook.
- мотивира потребителите , чрез предоставянето на риболовни турнири с възможност за награда.

Недостатъци:

- ограничение на данните за водоемите и турнирите до района на САЩ;
- ограничено използване на приложението до Андроид и IOS за използване на лаптоп е необходимо използването на допълнителни емулятори.
- няма възможност за работа офлайн (офлайн карта или дневник със записите).

1.1.3. Fishinda

Приложението предлага следните основни функционалности:

- възможност за изготвяне на риболовни дневници и правене на записи включващи (снимки, вид на екземпляра и допълнителна информация).
- предоставя информация за различните видове риба (снимка на рибата, дълбочина на кълване, предишни улови).
- бърз достъп до магазини предлагащи подходяща стръв за планирания вид риба.

Предимства:

- приложението е безплатно за изтегляне;

- възможност за комуникиране с останалите потребители със система подобна на Facebook.
- приложението е пригодено за използване по целия свят;
- има възможност за промяна на езика на интерфейса според нуждите на потребителя.
- приложението предлага купони за намаление на риболовни принадлежности предлагани в магазина му.

Недостатъци:

- ограничено използване на приложението до Андроид и IOS за използване на лаптоп е необходимо използването на допълнителни емулятори.
- няма възможност за работа офлайн (офлайн карта или дневник със записите).
- въпреки, че приложението е безплатно, без заплащане на месечния абонамент функционалността на приложението и силно ограничена.

1.1.4. FishTrack - Fishing Charts

Приложението предлага следните основни функционалности:

- ✓ предоставя информация за различните видове риба (снимка на рибата, дълбочина на кълване, предишни улови);
- ✓ възможност за изготвяне на риболовни маршрути с помощта на GPS;
- ✓ автоматично измерване и записване на влажността на въздуха и температура на околната среда и водата;
- ✓ предоставя сателитни и термични снимки на водата в даден регион.

Предимства:

- приложението е безплатно за изтегляне;
- може да бъде достъпвано както в интернет сайта им така и чрез мобилното им приложение;
- предлага запазването на офлайн снимки на маршрути и карти чрез мобилното си приложение.

Недостатъци:

- приложението е специализирано за риболов в солени води което ограничава използваемостта му;
- информацията в него не е оптимизирана за целия свят а само за определени държави.

1.1.5. Ribolovenatlas

Приложението предлага следните основни функционалности:

- ✓ предоставя информация за различните водоеми (местоположение, видове риби, историческа информация, екстри);
- ✓ визуализира карта на района;
- ✓ възможност за оставяне на коментари от потребителите относно водоемите;
- ✓ рейтинг система позволяваща на потребителите да оценят доколко един водоем е посещаван и харесван.

Предимства:

- приложението е бесплатно за използване;
- голяма част от информацията е достъпна без необходимостта от потребителска регистрация;

Недостатъци:

- сайта не е пригоден за разглеждане от мобилно устройство.
- предоставената информация на места е непълна и недава сведения за самите рибни видове.
- потребителите трябва да споделят преживяванията си и своите улови в коментарите вместо да има специално отредено място за тях.

1.1.6. Ribarnik.com

Приложението предлага следните основни функционалности:

- ✓ предоставя информация за различните водоеми (местоположение, видове риби, историческа информация);
- ✓ система за създаване на блогове;
- ✓ възможност за оставяне на коментари;
- ✓ рейтинг система.
- ✓ Онлайн магазин за риболовни принадлежности и стръв.
- ✓ Информационен сегмент съдържащ новини относно събития свързани с риболов.

Предимства:

- приложението е бесплатно за използване;
- голяма част от информацията е достъпна без необходимостта от потребителска регистрация;

- лесен за използване онлайн магазин със детайлна информация за стръвта и начините за използването и.
- интернет сайта е пригоден за разглеждане както на компютър така и във мобилна среда.

Недостатъци:

- предоставената информация на места е непълна и недава сведения за самите рибни видове.
- Потребителската информация се пази под формата на блокове което прави търсенето на конкретна капсулирана информация трудоемко.
- Основната цел на сайта е продажби чрез онлайн магазина , което на моменти прави сайта претрупан със нежелана реклама.
- Няма поощрителна система ,която да възнаграждава потребителите при споделяне на качествена информация.

1.1.7. Pro Angler Fishing App

Предимства:

- приложението е безплатно за използване;
- лесен за използване онлайн магазин със детайлна информация за стръвта и начините за използването и.
- Подробно ръководство за видове и техники за риболов.
- Морска прогноза в реално време, приливи и отливи.
- Споделяем дневник за улов.
- Всички големи крайбрежни и офшорни места за риболов във Флорида с актуални доклади от колеги риболовци.
- GPS с горещи точки за риболов.

Недостатъци:

- приложението е специализирано за риболов в солени води което ограничава използваемостта му;
- информацията в него не е оптимизирана за целия свят а само за определени държави.
- ограничено използване на приложението до Андроид и IOS за използване на лаптоп е необходимо използването на допълнителни емулятори.
- няма възможност за работа офлайн (офлайн карта или дневник със записите).
- въпреки, че приложението е безплатно, без заплащане на месечния абонамент функционалността на приложението и силно ограничена.

1.2. Обобщение от направения преглед на риболовни приложения

Повечето приложения са оптимизирани и пригодени за страни във и около САЩ и информацията за страните извън този кръг е ограничена. Някои от приложенията имат системи подобни на Facebook чрез които потребителите могат да споделят свой улов с други потребители на приложението, което изглежда излишно, защото повечето хора биха споделили своите успехи в риболова със свои приятели не дотолкова с непознати в приложението. Повечето приложения нямат възможност за работа офлайн и при по дълги риболовни излети това би могло да бъде недостатък. Повечето приложения са ограничени до използване от мобилен телефон и няма възможност за достъпване на информацията от други устройства като лаптоп, което би било по удобно за изготвяне на план за пътуването и приготвянето на оборудване и стръв. В повечето приложения детайлната информация за предишни улови и препоръчителна стръв са заключени зад месечен абонамент въпреки ,че повечето от тази информация е събрана от други потребители. Няма изготвен единен сайт за региона на България, който да дава информация за всички водоеми в страната, а съществуват множество сайтове които рекламират и дават информация само за собствения си регион. Много от сайтовете създадени с тази цел несе поддържат или са с остарял дизайн.

1.3. Цели, задачи и характеристики на разработката

Необходимо е създаването на приложение, оптимизирано за региона на България, с интерфейс на български език.

1.3.1. Основни цели

- ✓ Приложението трябва да може да бъде достъпвано от лаптоп за по лесно използване и изготвяне на план за риболовния излет от дома където в повечето случай се случва 90% от планирането и приготвянията.
- ✓ Информацията в приложението трябва да е безплатна като приходите за приложението не трябва да са обвързани със възможностите на потребителя да достъпва или използва с ограничения дадена функция.
- ✓ Потребителите трябва да могат да решават дали да споделят своята информация с останалите потребители или да я запазят само в профила си.
- ✓ Трябва да се създаде поощрителна система за потребителите който споделят своята информация, обогатявайки колективните знания за района, рибата и препоръчителната стръв.
- ✓ Приложението трябва да бъде оптимизирано за разглеждане от мобилно устройство , при нужда от информация за рибата или водоема в процеса на риболов.

- ✓ Модерация на постъпващите данни за отстраняване на ненужна или невярна информация.

1.3.2. Основни задачи

Задача 1

- Определяне на специфичните изисквания на приложението

Задача 2

- Определяне архитектурата на продукта.
- Модел на данните.

Задача 3

- Реализация на продукта.

1.3.3. Потребителски характеристики

Предвижда се реализацията на следните потребителски роли:

- нерегистриран потребител.
- регистриран потребител.
- регистриран потребител с ограничени функции (има достъп до системата, но не може да добавя или променя записи, коментари, предложения).
- регистриран потребител отговорен за верификация на данните постъпващи от потребителите (Модератор).
- регистриран потребител с роля управител на приложението(администратор).

2. Определяне на специфичните изисквания към програмния продукт

Тази глава съдържа списък на всички необходими функции, възможности и характеристики на системата.

2.1. Потребителски изисквания

2.1.1. Нерегистриран потребител.

- ✓ Нерегистриран потребител не трябва да има достъп до системата.
- ✓ Има достъп до наличната публична информация.

2.1.2. Регистриран потребител.

- ✓ Преглед на общодостъпната информация.
- ✓ Преглед на информацията споделена от други потребители.
- ✓ Въвеждане на данни за направен улов във личен риболовен дневник.
- ✓ Оставяне на коментар за даден вид риба, водоем, турнир.
- ✓ Предлагање на нови видове риба, водоеми, риби във водоеми.
- ✓ Участие в турнири.

2.1.3. Регистриран потребител с ограничени функции.

- ✓ Преглед на общодостъпната информация.
- ✓ Преглед на информацията споделена от други потребители.
- ✓ Преглед на личен риболовен дневник.

2.1.4. Регистриран потребител (Модератор).

- ✓ Одобрение на коректно направени потребителски записи.
- ✓ Премахване на некоректни споделени записи от потребители.
- ✓ Приемане или отхвърляне на предложения за добавяне на нов вид риба във колекцията.
- ✓ Приемане или отхвърляне на предложения за добавяне на нов водоем във колекцията.
- ✓ Приемане или отхвърляне на предложения за добавяне на съществуващ вид риба към съществуващ водоем.

2.1.5. Регистриран потребител – управител (администратор).

- ✓ Добавяне и премахване на роля (Модератор) към вече съществуващи потребители.
- ✓ Добавяне и премахване на роля (Администратор) към вече съществуващи потребители
- ✓ Премахване на потребители от тип (регистриран потребител).
- ✓ Добавяне на нови видове риби.
- ✓ Добавяне на нови водоеми.
- ✓ Добавяне на нови видове риболовна стръв.
- ✓ Добавяне на нови събития(Турнири).
- ✓ Прекратяване на турнир и награждаване на участниците.
- ✓ Обновяване на всички данни.

2.2. Изисквания към използвания модел

Системата трябва да бъде достъпна само за регистрирани потребители.

2.2.1. Част нерегистриран потребите.

Тази част е достъпна от всякакъв вид потребители, които не са регистрирани или не са се оторизирали със своите данни. Дава се достъп до базова информация и карта на водоемите.

2.2.2. Част регистриран потребител.

Тази част е достъпна за потребители, които са направили своята регистрация на сайта и са се оторизирали със своите потребителски данни за достъп.

2.2.2.1. Въвеждане на данни.

В този раздел регистрираният потребител трябва да има достъп и възможност за въвеждане на данни за:

- Риболовен дневник
 - Вид на уловената риба.
 - Размер.
 - Място на улов.
 - Използвана стръв.
 - Снимка на улова.
 - Дата и час на улова.

- Нов водоем
 - Име.
 - Географско положение.
 - Снимка.
 - Информация.

- Нов вид риба
 - Име.
 - Размер.
 - Снимка.
 - Информация.

- Нов вид риба за даден водоем
 - Риба.
 - Водоем.
 - Информация

- Коментари
 - За вид риба.
 - За водоем.
 - За турнир.

2.2.2.2. Промяна на данни.

В този раздел потребителят трябва да има възможност за:

- ✓ Премахване на лични записи от дневника.

2.2.2.3. Преглед на данните.

В този раздел потребителят трябва да има достъп до всички записи и коментари:

- ✓ Общодостъпни записи(базови записи за рибите в региона);
- ✓ Споделени записи(Записи направени и споделени от други потребители съдържащи по детайлно описание за направен улов);

2.3. Функционални изисквания

2.3.1. Потребител Class 1 – Регистриран потребител.

2.3.1.1. Функционално изискване 1.1.

ID: ФИ1

ЗАГЛАВИЕ: Посещаване на системата

ОПИСАНИЕ: Нерегистриран потребител не трябва да има достъп до информацията в системата.

ЦЕЛ: Системата трябва да е достъпна за ползване само след регистрация и автентикация.

ЗАВИСИМОСТ: не

2.3.1.2. Функционално изискване 1.2.

ID: ФИ2

ЗАГЛАВИЕ: Регистрация на потребител

ОПИСАНИЕ: Нерегистриран потребител трябва да има възможност за регистрация.

ЦЕЛ: Да се реализира възможност за регистрация.

ЗАВИСИМОСТ: ФИ1

2.3.1.3. Функционално изискване 1.3

ID: ФИЗ

ЗАГЛАВИЕ: Автентикация

ОПИСАНИЕ: Влизането на регистриран потребител в системата трябва да се осъществява чрез попълване и проверка на потребителско име и парола в login

страница.

При неуспешна автентикация потребителят трябва да се препраща на login страницата със съответното съобщение за неуспешен вход.

ЦЕЛ: Да се реализира възможност за проверка на входа в системата.

ЗАВИСИМОСТ: ФИ1, ФИ2

2.3.1.4. Функционално изискване 1.4.

ID: ФИ4

Заглавие: Регистриране на потребител

ОПИСАНИЕ: За да се регистрира всеки потребител трябва да попълни успешно формата за регистрация.

Сценарий: Необходима информация за регистрация

Осигуряване на възможност за регистрация на потребител чрез попълването на:

- Потребителско име;
- Име с което потребителя желае да се идентифицира в рамките на приложението;
- Парола на потребителя;
- Повторно въвеждане на парола;
- Имейл на потребителя;

Сценарий: Грешка при регистрация

Извеждане на съобщение за грешка при регистрация и възможност за нов опит при въвеждане на:

- вече регистрирано потребителско име;
- вече регистриран имейл адрес;
- въведените пароли не съвпадат

- типът на въведените данни неотговаря на изискванията на полето.
- пропуснато е попълването на някое от полетата;

2.3.1.5. Функционално изискване 1.5.

ID: ФИ5

Заглавие: Риболовен запис

ОПИСАНИЕ: За въвеждане на данните за направен улов да се използва опцията нов запис намираща се в менюто на риболовния дневник на потребителя. За разглеждане на записи се използва опцията преглеждане на записи.

Сценарий: Въвеждане на запис

При избиране на тази опция трябва да могат да се въведат данни за: вид на рибата, размер, място на улова, използвана стръв, дата и час на улова и допълнителна информация която потребителя сметне за удачна.

Сценарий: Поверителен запис

При създаването на риболовен запис потребителя има възможността да засекрети своя запис като така само той може да вижда детайлите около записа като това не премахва възможността записът да участва в турнири.

Сценарий: Грешка при създаване на запис

При некоректно попълване на полетата или непълна информация трябва да се извежда съобщение което да уведоми потребителя каква грешка е допуснал и как да я коригира.

Сценарий: Квалифициране за участие в турнир

При коректно попълване на всички полета и прикачване на валидна снимка потребителският запис придобива валидация, която го допуска до участие в турнири.

2.3.1.6. Функционално изискване 1.6.

ID: ФИ6

Заглавие: Получаване на информация.

ОПИСАНИЕ: Потребителя трябва да може да получава детайлна информация от системата за видовете риба намиращи се в даден водоем или за имената на водоемите в които се съдържа даден вид риба.

Сценарий: Необходима е информация за видовете риби за даден водоем

Потребителя избира от картата водоема от които се интересува и системата му предлага информация за видовете риба които могат да бъдат уловени в този водоем. Също така потребителя може да разгледа всички записи на улови направени в този водоем и споделени от други потребители.

Сценарий: Необходима е информация за водоемите в които може да бъде уловен даден вид риба.

Потребителя избира от списъка със всички видове риба записани в системата и получава информация за това в кои водоеми може да бъде намелена дадената риба.

2.3.1.7. Функционално изискване 1.7.

ID: ФИ7

Заглавие: Потребителски предложение.

ОПИСАНИЕ: Потребителя трябва да може да създава потребителски предложения чрез опциите:

Предлагане на нов вид риба,

Предлагане на нов водоем,

Предлагане на вид риба за даден водоем.

Сценарий: Предлагане на нов вид риба.

Потребителя попълва информация за нов вид риба включваща:

Име, максимален размер и тегло, информация за рибата и допълнителна информация за самото предложение.

Сценарий: Предлагане на нов водоем.

Потребителя попълва информация за нов водоем включваща:

Име, географско разположение , информация за водоема и допълнителна информация за самото предложение.

Сценарий: Предлагане на вид риба за даден водоем.

Потребителя избира вида риба и водоемът към които иска да бъде добавена и въвежда допълнителна информация разясняваща предложението му.

Сценарий: Коректно попълнена информация.

При коректно попълване на цялата информация съответното предложение се изпраща в списъка с предложения от същия вид и очаква разглеждане от модериращо лице.

Сценарий: Липса на снимка

При липса на снимков материал относно предложението се използва базова снимка съответстваща на типа на предложението .

Сценарий: Грешка при попълване на информацията

При некоректно или непълно въвеждане на информацията се извежда информативно съобщение до потребителя разясняващо проблема.

2.3.1.8. *Функционално изискване 1.8.*

ID: ФИ8

Заглавие: Турнири.

ОПИСАНИЕ: Потребителя може да получи информация относно даден турнир и да участва в турнирите със своите записи.

Сценарий: Получаване на списък с турнири

Чрез опцията турнири потребителят може да получи базова информация за протичащите в момента турнири като крайна дата, организатор и име на турнира.

Сценарий: Детайлна информация.

При избиране на даден турнир от списъка , потребителя получава по детайлна информация за него като начална и крайна дата, информация относно ограниченията за участие както и информация за наградите. Също така потребителя може да следи класирането за избрания турнир.

Сценарий: Участие в турнир.

Всички одобрени записи на потребителите участват автоматично във всички текущи турнири за които те покриват квалификационните критерии.

Сценарий: Награди.

Потребителите могат да получат награди в зависимост от своето класиране в протичащите турнири.

2.3.2. Потребител Class 2 – Модератор

2.3.2.1. *Функционално изискване 2.1*

ID: ФИ9

Заглавие: Вход в системата като модератор

ОПИСАНИЕ: За да може да регулира риболовните записи направени от други потребители , потребителят трябва да е регистриран като модератор.

Сценарий: Успешен вход в системата

За да има модераторски права потребителят трябва да е регистриран и да е влязъл в системата с акаунт на модератор.

2.3.2.2. Функционално изискване 2.2

ID: ФИ10

Заглавие: Управление на записи

ОПИСАНИЕ: Модератора трябва да може да премахва некоректни записи, и да приема коректни такива .

Сценарий: Премахване на запис

Модераторът може да премахва споделени риболовни записи , които не отговарят на изискванията или съдържат фалшива информация.

Сценарий: Одобрение на запис

Модераторът може да одобрява споделени риболовни записи , които отговарят напълно на изискванията и носят полезна информация в себе си.

2.3.2.3. Функционално изискване 2.3

ID: ФИ11

Заглавие: Управление на коментари

ОПИСАНИЕ: Модератора трябва да може да премахва коментари нарушаващи правилата на сайта или такива които не носят полезна информация.

Сценарий: Премахване на коментар

Модераторът може да премахва коментари направени от потребители ако тези коментари нарушават правилата на сайта, човешките права на останалите потребители , или просто не носят полезна информация във себе си (спам).

2.3.2.4. *Функционално изискване 2.4*

ID: ФИ12

Заглавие: Достъп до всички опции достъпни за всеки регистриран потребител.

ОПИСАНИЕ: Модераторът може да създава и въвежда риболовни записи в собствен дневник и да използва пълната функционалност на сайта като обикновен потребител.

2.3.3. Потребител Class 3 – Администратор

2.3.3.1. *Функционално изискване 3.1*

ID: ФИ13

Заглавие: Вход в системата с акаунт на управител (администратор)

ОПИСАНИЕ: За да може да управлява системата, потребителят трябва да е регистриран като управител.

Сценарий: Успешен вход в системата

За да има администраторски права потребителят трябва да е регистриран и да е влязъл в системата с администраторски акаунт.

2.3.3.2. *Функционално изискване 3.2*

ID: ФИ14

Заглавие: Управление на потребители

ОПИСАНИЕ: Управителят (администраторът) да има възможност да премахва потребители от сайта (временно или неограничено).

Сценарий: Премахван на потребител (временно)

Администратора може да забранява влизането на потребител във сайта за определен период от време при нарушение на правилата или спам.

Сценарий: Премахван на потребител (неограничено)

Администраторът може да премахва потребителски акаунти след многократни негативни проявления.

2.3.3.3. Функционално изискване 3.3

ID: ФИ15

Заглавие: Потребителски роли

ОПИСАНИЕ: Управителят (администраторът) да може да дава и премахва ролята на Администратор или Модератор на регистриран потребител.

Сценарий: Добавяне на Модератор

Администратора има възможността да предостави модераторски права на регистрирани потребители.

Сценарий: Добавяне на Модератор.

Администратора има възможността да предостави администраторски права на регистрирани потребители.

Сценарий: Премахване на привилегии

Администратора има възможността да премахва привилегировани роли от потребители като така ги връща до по ниско ниво на достъп.

2.3.3.4. Функционално изискване 3.4

ID: ФИ16

Заглавие: Потребителски предложения

ОПИСАНИЕ: Управителят (администраторът) може да одобрява или отхвърля потребителски предложения за обогатяване на базата данни.

Сценарий: Одобряване на предложение за добавяне на нов водоем.

Администратора има възможността да приеме потребителско предложение за нов водоем, като така предложения водоем и информацията за него стават част от базата данни с водоеми.

Сценарий: Одобряване на предложение за добавяне на нов вид риба.

Администраторът има възможността да приеме потребителско предложение за нов вид риба, като така предложението за нов вид риба и информацията за него стават част от базата данни с риби.

Сценарий: Одобряване на предложение за добавяне на вид риба към водоем.

Администраторът има възможността да приеме потребителско предложение за добавяне на вид риба към вече съществуващ водоем, като така предложението се създава връзка между двете в базата данни.

Сценарий: Отхвърляне на потребителско предложение

Администраторът може при неудовлетворени критерии или непълна информация за предложението да отхвърли предложението като така то се премахва от базата данни.

2.3.3.5. Функционално изискване 3.5

ID: ФИ17

Заглавие: Турнири

ОПИСАНИЕ: Управителя(администратора) може да добавя нови турнири , също така може да финализира награждаването след приключването на даден турнир.

Сценарий: Създаване на турнир

Администратора има възможността да създава нови турнири като

Определя техните критерии за участие: начална дата ,крайна дата, желан вид риба, ограничение относно водоема и също така да поясни детайли около награждаването.

Сценарий: Финализиране на турнир.

Администратора може при изтекъл период за участие в даден турнир да финализира и затвори турнира и да раздаде наградите на участвалите потребители на съответните позиции.

2.4. Нефункционални изисквания

2.4.1. Изисквания за сигурност на системата

2.4.1.1. Нефункционално изискване 1.1

ID: НФИ1

Заглавие: Защита от неоторизиран достъп.

ОПИСАНИЕ: Необходимо е да се осигури защита от неоторизиран достъп. За целта достъпът трябва да се извършва чрез потребителско име и парола, след регистрация на потребител и получаване на съответните права.

2.4.1.2. Нефункционално изискване 1.2

ID: НФИ2

Заглавие: Права за достъп на потребителите.

ОПИСАНИЕ: Правата за достъп трябва да бъдат определяни на база на потребителските роли. Всеки потребител може да има само една роля.

2.4.1.3. Нефункционално изискване 1.3

ID: НФИЗ

Заглавие: Продължителност на потребителска сесия

ОПИСАНИЕ: Потребителската сесия трябва да се прекъсва след определен период от време на неактивност.

2.4.1.4. Нефункционално изискване 1.4

ID: НФИ4

Заглавие: Защита на отделните части на системата

ОПИСАНИЕ: Потребители без съответните права не трябва да имат достъп до конкретните части от системата. За целта при достъп до даден раздел трябва да се извършва проверка на текущата потребителска сесия.

2.4.2. Изисквания към потребителския интерфейс

2.4.2.1. Нефункционално изискване 2.1

ID: НФИ5

Заглавие: Език на потребителския интерфейс

ОПИСАНИЕ: Потребителският интерфейс трябва да е на български език.

2.4.2.2. Нефункционално изискване 2.2

ID: НФИ6

Заглавие: Дизайн на потребителския интерфейс

ОПИСАНИЕ: Дизайнът на потребителския интерфейс трябва да е изчистен, да е опростен и лесен за използване от страна на потребителите. Да се осигури безпроблемна ориентация и навигация между отделните части и раздели на системата.

2.4.3. Изисквания към потребителския интерфейс

2.4.3.1. Нефункционално изискване 3.1

ID: НФИ7

Заглавие: Архитектура на уеб приложението

ОПИСАНИЕ: Уеб приложението трябва да бъде изградено на базата на трислойна архитектура и да поддържа работа с база от данни.

2.4.3.2. Нефункционално изискване 3.2

ID: НФИ8

Заглавие: Работа с различни ОС и СУБД

ОПИСАНИЕ: Системата трябва да работи независимо от използваните операционни системи и СУБД.

2.4.3.3. Нефункционално изискване 3.3

ID: НФИ9

Заглавие: Достъпност и бързодействие

ОПИСАНИЕ: Системата трябва да бъде достъпна 24 часа в денонощието, 7 дни в седмицата. Да се осигури бързодействие на приложението.

2.4.3.4. Нефункционално изискване 3.3

ID: НФИ10

Заглавие: Съвместимост с различни уеб браузъри

ОПИСАНИЕ: Системата трябва да работи пълноценно с най-популярните уеб браузъри:

- Microsoft Edge 93 и по-нови версии;
- Mozilla Firefox 91.0 и по-нови версии;
- Google Chrome 93.0 и по-нови версии;
- Internet Explorer 11.0 и по-нови версии.
- Opera 78.0 и по-нови версии;

3. Архитектура на продукта и модел на данните

3.1. Избор на средства за реализация

3.1.1. Програмен език

Първата стъпка при разработването на приложение е изборът на програмен език . За реализиране на нашето приложение ще използваме програмния език. Някои от предимствата на езика са :

- Java е платформено независим – кодът се компилира до байт код, а изпълнението на програмата се извършва чрез виртуална машина (Java Virtual Machine). Това осигурява лесна преносимост между различни софтуерни или хардуерни платформи, т.е. чрез виртуалната машина Java приложение може да работи независимо от архитектурата или от операционна система, използвани от даден компютър[6].
- Java е обектно-ориентиран език за програмиране което го прави обектите в него лесно съпоставими с обекти от нашето ежедневие. Също така кодът има висока степен на преизползваемост.
- Java е език за програмиране от високо ниво, който позволява абстракция на семантиката на изпълнение на компютърна архитектура от спецификацията на програмата. Тази абстракция прави процеса на разработване на програма много по-прост и разбираем .

3.1.2. Визуализация

За визуализация на уеб страниците ни ще използваме шаблонната система Thymeleaf.

Thymeleaf е шаблонна server-side система за уеб разработка както и за самостоятелни среди.

Основната цел на Thymeleaf е да внесе елегантни естествени шаблони в процеса на разработка – HTML код, който може да се показва правилно в браузърите и също така да работи като статични прототипи, което позволява по -силно сътрудничество в екипите за разработка. [7].

3.1.3. Модел на БД

Изборът на релационен модел за базата данни е заради неговата висока скорост на производителността. Релационният модел позволява обработка на големи порции от данни с единична операция.

Като система за управление на бази данни избираме MySQL. Сред предимствата на тази СУБД са: отворен код, наличие на безплатен лиценз, поддръжка на интерфейси за достъп от множество програмни езици, както и работа с различни ОС.

Структурата на релационна база данни, като MySQL, може да се формира чрез определяне на субектите, участващи в системата и отношенията, които ги свързват едно с друго.[8]

3.1.4. Архитектура

Уеб приложението е изградено на основата на трислойна архитектура, с прилагане на шаблона Модел-Изглед-Контролер. Използването на подобна архитектурата осигурява разделяне на бизнес логиката от графичния интерфейс и данните в приложението. „Model-View-Controller“ („MVC“) е архитектурен шаблон, който най-често се използва при създаването на потребителски интерфейс. Той „разделя“ приложението на три взаимосвързани части. Това е направено с цел да се раздели вътрешното представяне на информация от начините по които информацията се представя на и приема от потребителя. MVC шаблонът разделя тези главни компоненти, което позволява на разработчиците да използват отново вече написан код по-ефективно, а също така позволява и паралелна разработка. [9].

В дадения проект MVC шаблона е реализиран в няколко пакета:

- Model
- Repository
- Service
- Web
- Dto

Предимства:

Тъй като MVC разделя основните компоненти на приложението, това позволява на разработчиците да работят паралелно по различни компоненти, без да оказват влияние или да си пречат един на друг. Създавайки компоненти, които са независими едни от други, разработчиците могат да използват многократно един компонент в различни приложения. Изгледът на едно приложение може да се преработи бързо и лесно за друго сходно приложение, което борави с коренно различна структура от данни; моделът който обработва данните и който представлява цялостната структура е отделен компонент спрямо изгледа и това дава свобода на разработчиците да използват наново един и същи код, според нуждите си.

3.1.4.1. *Моделът (Model)*

„Моделът (Model)“ е централен компонент в шаблона. Това динамичната структура от данни на приложението, независима от потребителския интерфейс. Моделът управлява данните, логиката и правилата на приложението.

Моделът в реализирания проект се състои от класовете, реализирани в пакетите Model, Repository, Service и Dto .

Пакетът Model съдържа класовете, реализиращи Plain Old Java Objects (POJO) обектите. Всеки клас в този пакет съдържа само член-променливи и съответните методи за достъп (get) и методи за промяна (set).

Пакетът Repository съдържа репозиторията ,които разширяват JPA repository в тях са написани методи за достъп до базата данни а JPA прави тяхната имплементация, в разглежданото приложение заявките са написани чрез конвенцията на JPA за наименоуване на методи. Чрез използването на специален синтаксис при наименоуване на репозитории методите, тя конвертира имената във заявка.

Пакетът Service съдържа бизнес логиката на приложението, като интерфейсите Service предоставят методите за работа а ServiceImpl имплементират бизнес логиката зад тези методи. Те са тези ,които извикват методите на съответните им репозиторията. Така се постига по голямо ниво на абстракция между методите на сървисите и тяхната имплементация.

Пакетът Dto съдържа (Data Transfer Object) - Обекти, които пренасят данни между процесите, за да се намали броя на извикванията на методи. Друго предимство е капсулирането на механизма за сериализация за прехвърляне на данни. Като капсулират сериализацията по този начин, DTO пазят логиката от останалия код и също така предоставят ясна точка за промяна на сериализацията, ако е необходимо.

При сериализация на обекти, съдържащи в себе си двустранни релационни връзки, към JSON формат могат да се получат безкрайни рекурсии ако сериализацията не бъде контролирана.

3.1.4.2. *Изглед*

Изглед (View) е изходящият поток от информация (това, което приложението изпраща като отговор до дисплея, респективно – до потребителя, в следствие на неговата заявка). Възможни са няколко различни изгледа на една и съща информация,

като например различни диаграми за мениджмънт на даден ресурс или различни таблици.

Използвайки депендънсита `spring-boot-starter-thymeleaf` получаваме автоматични настройки за `thymeleaf` които се справят със свързването на имената на шаблоните, подадени от контролера към реалните обекти на изгледите.

Шаблоните на проекта се намират във папката `Resources->Templates` което е папката за шаблони по подразбиране.

3.1.4.3. Контролер

„Контролерът (Controller)“ е третата част от този шаблон. Той приема потребителския вход (т.е. данните, които потребителя въвежда, неговите заявки и т.н.) и ги преобразува в команди към модела или изгледа. Той е реализиран чрез отделни контролери в пакета `Web`.

Контролерът трябва да съдържа в себе си само препратки към изгледи или сървиси и възможно най-малко бизнес логика.

Контролерът не оказва влияние върху данните, той само извиква методи от Модела – вика данните от Модела и извиква допълнителни методи върху Модела, който предварително обработва данните, след което ги предоставя на Изгледа.

Контролерът не визуализира информация, а предоставя информация на Изгледа, който визуализира данните.

3.1.5. Модел на данните

За достъп до релационната база данни се използва обектно-ориентиран подход. При реализацията на проекта се използва релационна база от данни `MySQL`, както и същият сървър за бази данни. Връзката между базата данни и приложението се осъществява с използването на `Spring Data JPA` и драйвера `MySQL Connector/J 5.1.44`.

Принципът на работа е следният – Когато е необходимо обръщение към СУБД , някой от класовете за извършване на бизнес логиката се обръща към съответното репозитори , което от своя страна чрез `JPA` превръща името на извикания метод във `SQL` заявка която се изпраща към базата.

3.1.6. Детайлно описание на използваните таблици

3.1.4.1. Таблица Стръв (Bait)

Тази таблица се състои от следните полета:

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- name с тип varchar и размер 255 (това е поле което съхранява името на примамката).

3.1.4.2. Таблица Риба (Fish)

Тази таблица се състои от следните полета:

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- info с тип text (това е поле което ще съдържа допълнителна информация за рибата, като например отличителни черти)
- name с тип varchar и размер 255 (това е поле което съхранява името на вида риба)
- max_size с тип double (това е поле което съхранява максималната възможна ширина на рибата измерена в сантиметри)
- max_weight с тип double (това е поле което съхранява максималното възможно тегло за рибата измерено в килограми)
- path с тип varchar и размер 255 (това е поле което съхранява пътят до местоположението на снимка на дадената риба)

3.1.4.3. Таблица Коментари (Comments).

Тази таблица се състои от следните полета:

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- body с тип text (това е поле което ще съдържа текста с коментара на потребителя, който носи главното значение на изразеното от потребителя мнение)

- parentId с тип bigint (това поле пази информацията за идентификационния номер на родителския елемент на коментара)
- parentType с тип varchar и размер 255 (това поле пази информацията за типа на родителския елемент към когото коментара принадлежи и заедно с parentId подпомага за правилното идентифициране на този елемент)
- userId с тип bigint (това е външен ключ сочещ към идентификационния номер на потребителя, създал коментара)
- date с тип dateTime (това поле пази информация за датата на създаване на коментара)

3.1.4.4. Таблица Риболовни записи (Journal).

Тази таблица се състои от следните полета:

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- date с тип dateTime (това поле пази информация за датата на риболовния запис)
- is_eligible_for_tourney с тип Boolean (това поле не дава информация дали риболовният запис отговаря на изискванията за допускане до турнир)
- path с тип varchar и размер 255 (това е поле което съхранява пътят до местоположението на снимка за риболовния запис)
- shared с тип Boolean (това поле не дава информация дали потребителят създал записа желае да го сподели с останалите потребители или иска той да бъде поверителен)
- size с тип double (това е поле което съхранява ширината на уловената рибата измерена в сантиметри)
- weight с тип double (това е поле което съхранява теглото на уловената рибата измерено в килограми)
- baitId с тип bigint (това е външен ключ сочещ към идентификационния номер на стръвта използвана при улова)
- waterId с тип bigint (това е външен ключ сочещ към идентификационния номер на водоема на който е уловена рибата)
- fishId с тип bigint (това е външен ключ сочещ към идентификационния номер на уловения вид риба)
- userId с тип bigint (това е външен ключ сочещ към идентификационния номер на потребителя, създал записа)
- is_moderated с тип Boolean (това поле не дава информация дали записът е бил одобрен от модериращо лице)

3.1.4.5. Таблица Предложения за вид риба (fishProposals).

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- fish_info с тип text (това е поле което ще съдържа допълнителна информация за рибата, като например отличителни черти)
- info с тип text (това е поле което ще съдържа допълнителна информация за предложението, която да бъде взета предвид при оценка на предложението)
- max_size с тип double (това е поле което съхранява максималната възможна ширина на рибата измерена в сантиметри)
- max_weight с тип double (това е поле което съхранява максималното възможно тегло за рибата измерено в килограми)
- path с тип varchar и размер 255(това е поле което съхранява пътят до местоположението на снимка на дадената риба)
- name с тип varchar и размер 255 (това е поле което съхранява името на вида риба)
- userId с тип bigint (това е външен ключ сочещ към идентификационния номер на потребителя, създал предложението).

3.1.4.6. Предложения за добавяне на риба към водоем (fishToWaterProposals).

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- userId с тип bigint (това е външен ключ сочещ към идентификационния номер на потребителя, създал предложението)
- info с тип text (това е поле което ще съдържа допълнителна информация за предложението, която да бъде взета предвид при оценка на предложението)
- fish_Id с тип bigint (това е външен ключ сочещ към идентификационния номер на рибата за която се отнася предложението)
- water_Id с тип bigint (това е външен ключ сочещ към идентификационния номер на водоема за който се отнася предложението).

3.1.4.7. Потребителски роли (role).

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)

- name с тип varchar и размер 255 (това е поле което съхранява името на дадената роля).

3.1.4.8. Турнири (tournament).

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- end_date с тип dateTime (това поле пази информация за крайната дата на турнира)
- host_name с тип varchar и размер 255 (това е поле което съхранява името фирмата или човека организатор на турнира)
- name с тип varchar и размер 255 (това е поле което съхранява името турнира)
- rewards_info с тип varchar и размер 255 (това е поле което съхранява информация за награждаването на турнира)
- starting_date с тип dateTime (това поле пази информация за началната дата на турнира)
- status с тип varchar и размер 255 (това е поле което съхранява информация етапа на развитие на турнира като възможните състояния са: "Активен", "Затворен" и "Завършил")
- waterId с тип bigint (това е външен ключ сочещ към идентификационния номер на водоема ,ако такъв е избран, със който е обвързан турнира)
- fishId с тип bigint (това е външен ключ сочещ към идентификационния номер на рибата ,ако такава е избрана, със която е обвързан турнира).

3.1.4.9. Награди от турнири (tournament_rewards).

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- monetary_value с тип double (това е поле което съхранява максималната възможна ширина на рибата измерена в сантиметри)
- name с тип varchar и размер 255 (това е поле което съхранява името на наградата)
- placing с тип int (полето съхранява позицията на класирания се в турнира за която се отнася наградата)
- tournament_id с тип bigint (това е външен ключ сочещ към идентификационния номер на турнира с който е обвързана наградата)

- user_id с тип bigint (това е външен ключ сочещ към идентификационния номер на потребителя притежател на наградата, който се назначава след приключване на турнира).

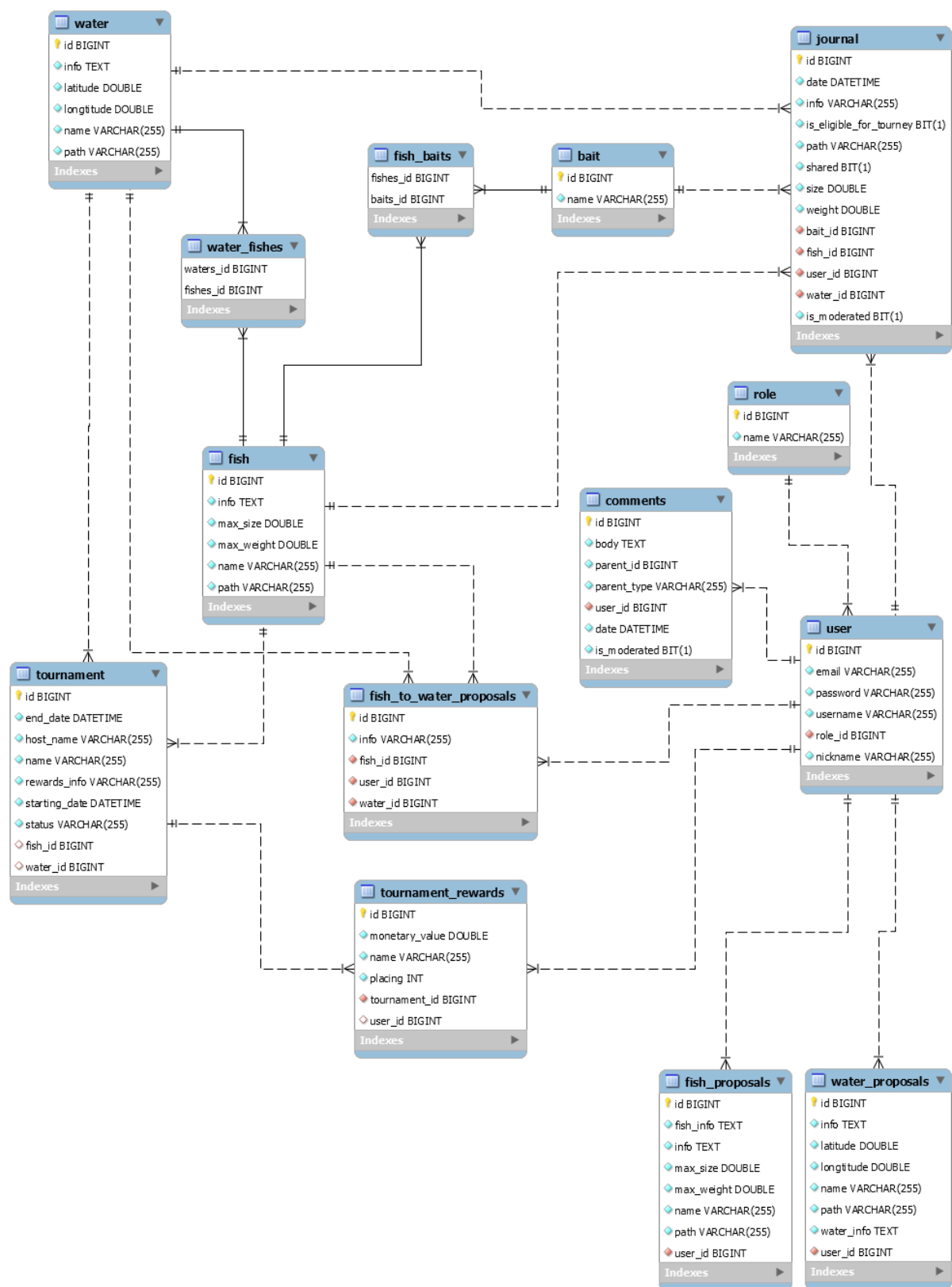
3.1.4.10. Потребители (user)

- id с тип bigint (това е първичен ключ, който е уникален и ни служи за връзка с другите таблици)
- email с тип varchar и размер 255 (това е поле което съхранява имейла на потребителя)
- password с тип varchar и размер 255 (това е поле което съхранява паролата на потребителя)
- username с тип varchar и размер 255 (това е поле което съхранява потребителското име на потребителя използвано за идентификация в системата)
- nickname с тип varchar и размер 255 (това е поле което съхранява псевдоним или име на потребителя използвано за представянето му пред останалите потребители)
- role_id с тип bigint (това е външен ключ сочещ към идентификационния номер на ролята която потребителя притежава)

3.1.4.11. Водоему (water)

- name с тип varchar и размер 255 (това е поле което съхранява името на водоема)
- info с тип text (това е поле което ще съдържа допълнителна информация за водоема, като например удобни зони за риболов)
- latitude с тип double (това е поле което съхранява географската ширина на точка от водоема)
- longitude с тип double (това е поле което съхранява географската дължина на точка от водоема)
- path с тип varchar и размер 255 (това е поле което съхранява пътят до местоположението на снимка на водоема).

3.1.4.12. Цялостна архитектура на използваната БД.



Фигура 1

4. Функционалност на разработеното уеб приложението

4.1. Програмна реализация

Една от главните функционалности на приложението е възможността за получаване на географска информация за различните водоеми с помощта на карта и маркери отбелязващи местоположението на водоемите. Потребителя трябва да получава визуална информация за разположението на желания водоем върху територията на България, приблизителните размери и форма на водоема спрямо средата, а също така и реален изглед на водоема под формата на фотографска снимка. Също така при маркиране на даден водоем потребителя трябва да получава информация за различните видове риба, намиращи се във водоема.

За целта са използвани функционалностите предоставени от библиотеката на OpenLayers.

OpenLayers е модулна, високопроизводителна, функционална библиотека за показване и взаимодействие с карти и геопространствени данни.

OpenLayers библиотеката е с отворен код и може да се използва за показване на картографски данни в уеб браузъри.

Библиотеката се предлага с вградена поддръжка за широк спектър от търговски и безплатни източници на изображения и векторни плочки и най-популярните отворени и патентовани формати на векторни данни. [10]

За визуализацията на самата карта е необходимо основния и слой, който е реалната географска карта, и допълнителните слоеве (векторен слой със всички маркери указващи местоположението на водоемите) да бъдат инициализирани със желаните данни.

За целта една от първите стъпки е да вземем данните от БД за всички водоеми съхранени в системата и да ги предадем към Javascript кода, с който ще конфигурираме картата и нейните слоеве.

```
1. @GetMapping("/{home"})
2. public String home(Model model) {
3.     model.addAttribute("waters", waterService.findAll());
4.     model.addAttribute("fishList", new ArrayList<Fish>());
5.     return "home";
6. }
```

Чрез get контролера, отговарящ за генериране и инициализиране на началната страница, подаваме списък със всички налични водоеми в БД като за това извикваме сървис отговарящ за тази задача. Списъкът с водоеми подаваме като атрибут на

модела, който ще бъде достъпен от нашето „view“ , “home.html” . Заедно с това подаваме и празен списък от риби, който е необходим за първоначалното инициализиране на thymeleaf елемента отговарящ за попълването на таблицата с риби във , “home.html”.

За да можем да вземем списъка с водоеми и той да е достъпен в нашия javascript код е необходимо до използваме синтаксиса на thymeleaf:

```
1. <script th:inline="javascript">
2.
3. var waters = [{${waters}}];
4.
5. </script>
```

Така при зареждане на страницата thymeleaf взима стойността намираща се в „waters“ атрибута и я подава на променливата.

Следващата стъпка е да преобразуваме необходимите данни за водоемите до обекти от тип „feature“ , които са градивен елемент за векторните слоеве на OpenLayers.

```
1. var features = [];
2. for (var i=0; i<waters.length;i++) {
3.     features.push(coloredSvgMarker([waters[i].longitude,
waters[i].latitude], waters[i].name, i+1));
4. }
```

Чрез обикновен цикъл взимаме информацията за географското разположение на водоемите, тяхното име, а също така и брояч с които ще идентифицираме водоема и го подаваме към функцията чрез която ще ги преобразуваме в необходимите обекти.

```
function coloredSvgMarker(lonLat, name, place) {
2.
3.     var feature = new ol.Feature({
4.         geometry: new ol.geom.Point(ol.proj.fromLonLat(lonLat)),
5.         name: name,
6.         id:place
7.     });
8.
9.
10.
11.     feature.setStyle(
12.         new ol.style.Style({
13.             image: new ol.style.Icon({
14.                 anchor: [0.5, 46],
15.                 anchorXUnits: 'fraction',
16.                 anchorYUnits: 'pixels',
17.                 src: '../images/fishmark.png'
18.             })
19.         })
20.     );
21.     return feature;
22. }
```


Функцията „coloredSvgMarker“ взима получените параметри и създава необходимият ни обект от тип „ol.Feature“ със тях . Чрез „setStyle“ метода създаваме самите маркери обвързани със този векторен обект , задава размерите на маркера както и изображението, което потребителя ще определя като маркер. След това връщаме обекта обратно в нашия масив „features“ .

След като имаме градивните елементи на векторния слой , създаваме самия векторен слой.

```
1. var vectorSource = new ol.source.Vector({ // VectorSource({
2.     features: features
3. });
4.
5. var vectorLayer = new ol.layer.Vector({ // VectorLayer({
6.     source: vectorSource
7. });
8.
```

С масива от обекти създаваме сорс за нашия векторен слой , които е необходимо да бъде обект от тип „ol.source.Vector“ и след това създаваме самия векторен слой от тип „ol.layer.Vector“ като му подаваме по рано създадения сорс.

Друг важен слой за нашата карта е прозорчето съдържащо снимка и името на водоема, което ще се появява всеки път когато потребителя кликне върху маркера на някой от водоемите.

```
1. var overlay = new ol.Overlay({
2.     element: container,
3.     autoPan: true,
4.     autoPanAnimation: {
5.         duration: 250
6.     }
7. });
```

Създаваме html елементите, който ще съдържат и изобразяват снимката на водоема и неговото име и съхраняваме идентификацията указваща този елемент в променливата „container“ . За създаване на този повърхностен слой е необходимо да създадем обект от тип „ol.Overlay“ който позволява елементи да бъдат обвързани с картата , но също така могат да бъдат променяни и премахвани динамично без да променят съдържанието на самата карта. Това е необходимо за да можем да преизползваме един и същи елемент за визуализиране на всичките водоеми.

След като сме се погрижили за векторния слой отговарящ за маркерите можем да започнем създаването на самата карта. Първото нещо което ще ни е необходимо е да определим границите на нашата карта , което ще отговаря на това до колко потребителят може да се придвижва по картата. За нашата цел е необходимо потребителя да може да преглежда целия регион на България, както и водоемите по

границите с останалите държави без да има възможност да излиза далеч от тези граници за да се избегне объркване и да се спестят ресурси от зареждане на неизползваните части.

```
1. const bulgariaExtent = ol.proj.transformExtent([22.345023234,  
41.238104147, 28.603526238, 44.228434539], 'EPSG:4326', 'EPSG:3857');
```

Създаваме обект от тип „ol.proj.transformExtent“ който ще съхранява желаните от нас гореспоменати граници под формата на максимална и минимална географска дължина и максимална и минимална географска ширина, а също така задаваме формата в който ще се трансформират координатите за да могат да бъдат прочетени от openLayers.

```
1. var map = new ol.Map({  
2.   layers: [  
3.     new ol.layer.Tile({ // TileLayer({  
4.       preload: Infinity,  
5.       source: new ol.source.OSM()  
6.     }), vectorLayer  
7.   ],  
8.   overlays: [overlay],  
9.   target: 'map',  
10.  view: new ol.View({  
11.    center: ol.proj.fromLonLat([25.4833, 42.7250]),  
12.    zoom: 6,  
13.    maxZoom: 15,  
14.    minZoom: 7,  
15.  extent: ol.extent.buffer(bulgariaExtent, 80000),  
16.    showFullExtent: true,  
17.  })});
```

Създаваме обект от тип „ol.Map“ които ще представлява нашата карта и задаваме желаните от нас характеристики. Добавяме вече създаденият от нас векторен слой съдържащ маркерите на водоемите. Задаваме създадения от нас повърхностен слой съдържащ прозореца с снимка и име на водоема . Поставяме координатите на в началното разположение на изгледа спрямо картата за да може България да е центрирана като изглед при първоначално стартиране на страницата. Задаваме настройки за начално увеличение на изгледа на картата, максималното увеличение ,което потребителя може да постигне и минималното увеличение. Накрая указваме създадените по-рано от нас граници на картата.

Последната стъпка е да подадем нашият обект съдържащ желаната от нас карта на елемент който да я визуализира.

4.2. Ръководство на потребителя

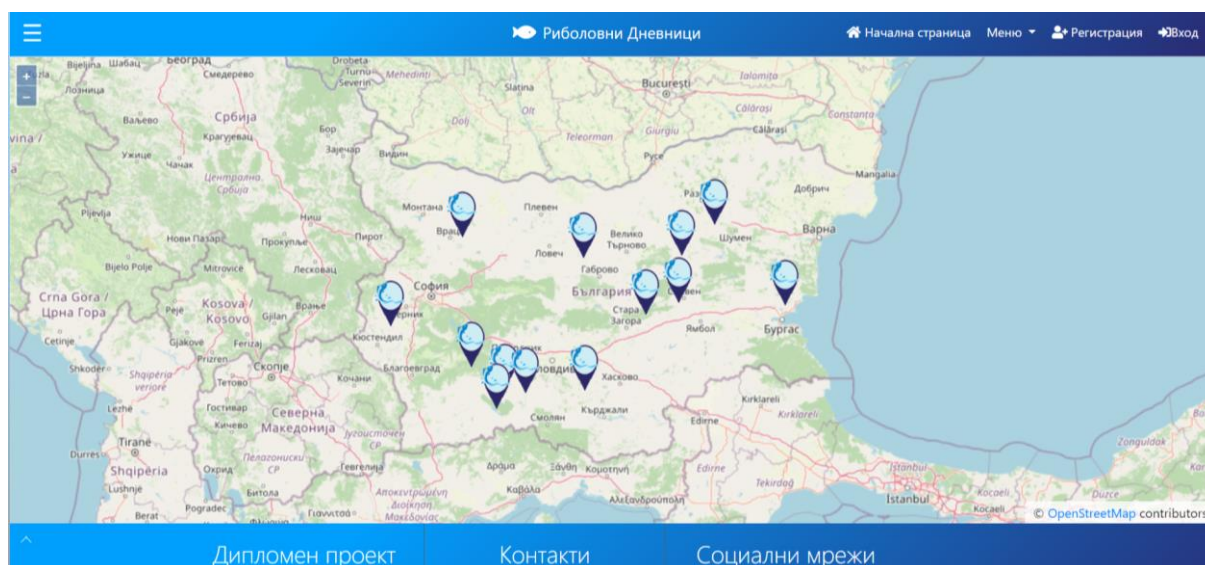
Ръководството на потребителя е съществена част за правилната употреба на даден софтуерен продукт. Той дава информация как по най – добър начин той да се използва. Също така дава информация за хардуерните и софтуерните изисквания към машината, на която се стартира.

Единственото изискване към потребителя е да притежава устройство с достъп до интернет. За достъп до приложението трябва да се въведе неговия адрес в полето на брауъра.

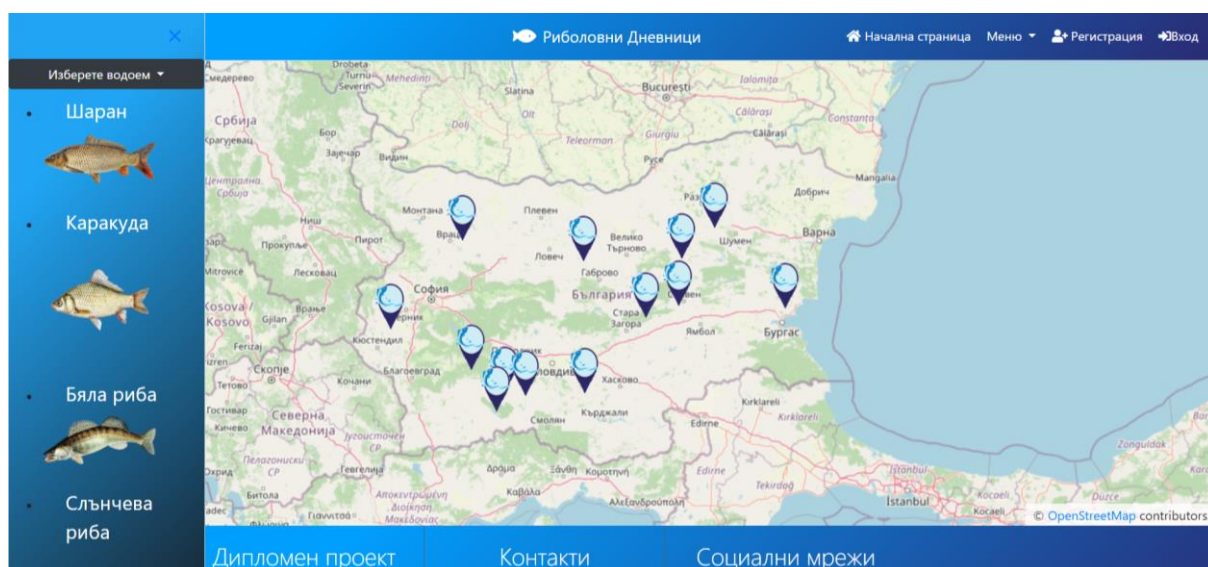
При първоначално зареждане на сайта пред потребителя се отваря началната страница на приложението „/home“ на нея се намира основната част на функцията за картографиране на водоемите а именно интерактивна карта на региона на България със маркери указващи отделните водоеми.

На страницата се намира и хедър съдържащ бутони със връзка към останалите функционалности на приложението. Също така и футър съдържащ допълнителна информация като контакти и социални мрежи.

На фиг. 4.1 може да се види началната страница на приложението съдържаща карта, хедър и футър а на фигури 4.1а и 4.1б са показани разгърнати съответно страничното меню и футъра.

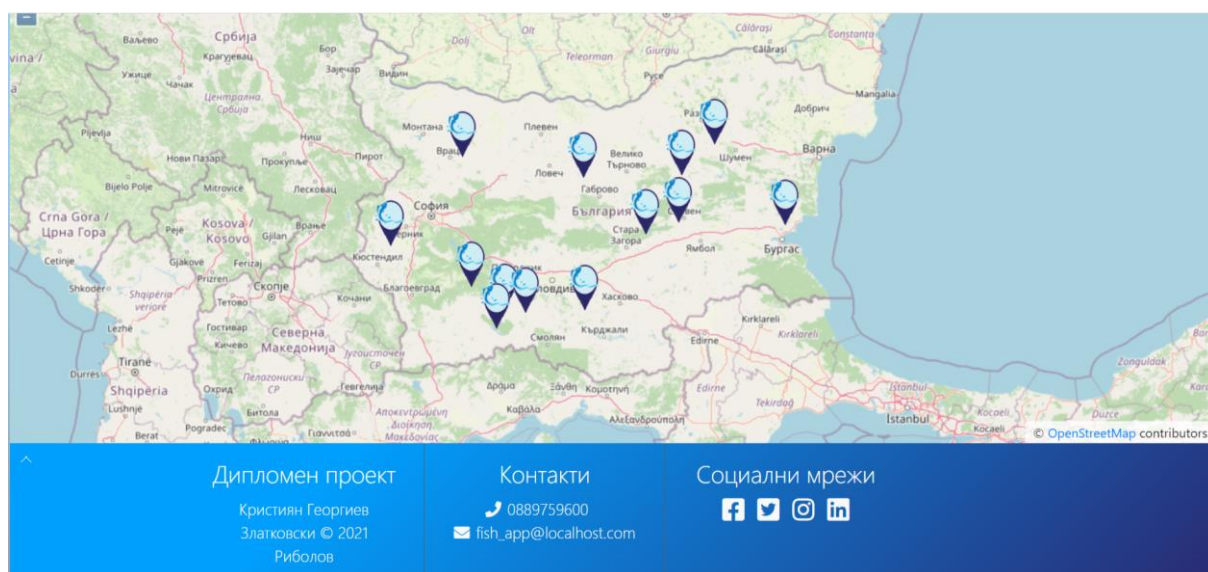


Фиг. 4.1 Начална страница



Фиг. 4.1а Странично меню

Страничното меню съдържа всички видове риби характерни за даден водоем, може да бъде разгърнато чрез бутон намиращ се върху лявата част на хедъра. Също така чрез падащ списък намиращ се върху страничното меню могат да бъдат филтрирани водоемите чрез тяхното име.



Фиг. 4.16 Хедър на началната страница

Футъра може да бъде разгърнат , чрез стрелка намираща се в левият му ъгъл. В него се съдържа кратка информация за разработката, контакти за свързване с администратора на приложението и различни социални мрежи свързани със приложението. Всички бутони са активни ,като тези от контактите копират дадената информация в буфера за по-нататъшно използване , а тези за социалните мрежи служат за препратка към съответния сайт.

На фиг. 4.2 може да се види страницата със формата за регистрация ,която се грижи за създаването на потребителски профили и валидация на въведените данни.

The image shows a registration form overlaid on a background of a wooden pier with several sailboats. The form is titled 'Създайте свой профил' (Create your profile). It contains five input fields: 'ПОТРЕБИТЕЛСКО ИМЕ' (Username), 'ПСЕВДОНИМ' (Nickname), 'ИМЕЙЛ' (Email), 'ПАРОЛА' (Password), and 'ПОТВЪРДЕТЕ ПАРОЛАТА' (Confirm password). Below these fields is a blue button labeled 'Изпращане' (Send). The website's header is visible at the top, showing the logo 'Риболовни Дневници' and navigation links: 'Начална страница' (Home), 'Меню', 'Регистрация', and 'Вход'.

Фиг. 4.2 Регистрационна форма

Потребителя въвежда своята информация и след валидиране на тази информация от страна на системата резултатът може да бъде два вида. Ако всичката информация е въведена коректно и избраните потребителско име и парола са свободни , на потребителя се създава профил със въведените от него данни. При грешка в попълването на някое от полетата, оставяне на някое от полетата празно или вече съществуващи потребителско име или имейл, потребителят се връща към полето за регистрация и получава информация за допуснатите от него грешки.

На фиг. 4.3 може да се види страницата със формата за вход в системата .

The image shows a login form overlaid on a background of a wooden pier extending into a calm body of water with a boat in the distance. The form is titled 'Моля влезте в системата' (Please log in to the system). It contains two input fields: 'Потребителско име' (Username) and 'Парола' (Password). Below these fields is a blue button labeled 'Вписване' (Login). Below the login button is a link that says 'или Създаване на профил' (or Create profile). The website's header is visible at the top, showing the logo 'Риболовни Дневници' and navigation links: 'Начална страница' (Home), 'Меню', 'Регистрация', and 'Вход'.

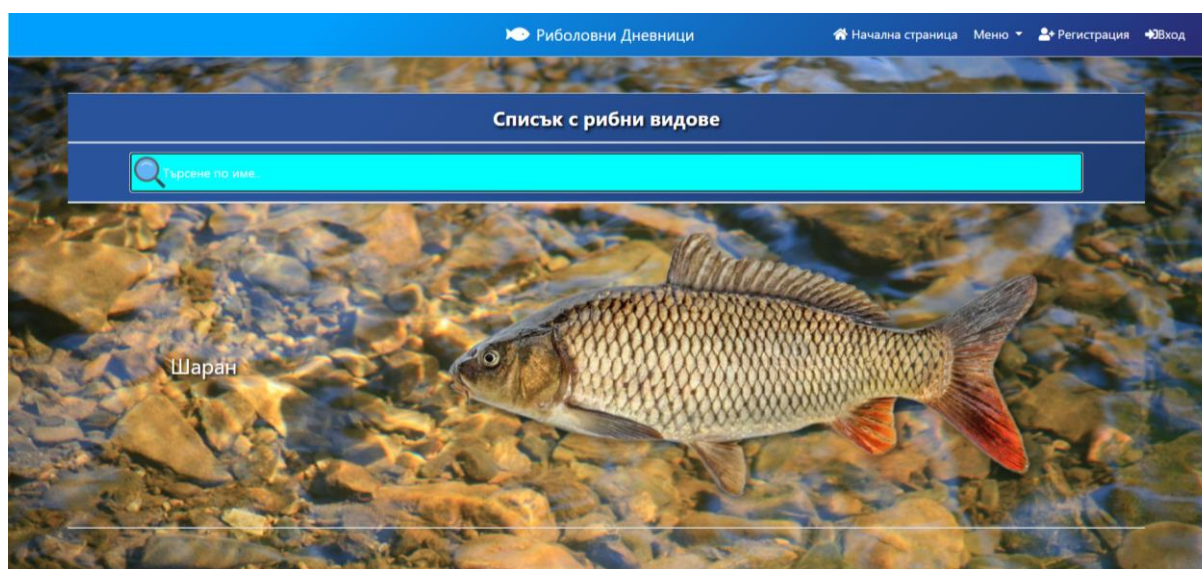
Фиг. 4.3 Страница за потребителски вход

Страницата за вход се грижи за въвеждане на потребителя в системата чрез валидация на въведените от него потребителски данни . При правилно попълнени потребителско име и парола, които съществуват в базата данни за потребители, се допуска вход в системата.

При несъществуващи име и парола или разминаване между въведените данни и тези в БД , потребителя се връща на страницата за вход и получава съобщение за направените от него грешки.

Потребителите, които не са влезли в системата имат достъп само до началната страница и двете функционалности за разглеждане на риби и водоеми , линка за които се намира в падащото меню.

На фиг. 4.4 може да се види страницата съдържаща всички рибни видове налични в системата.

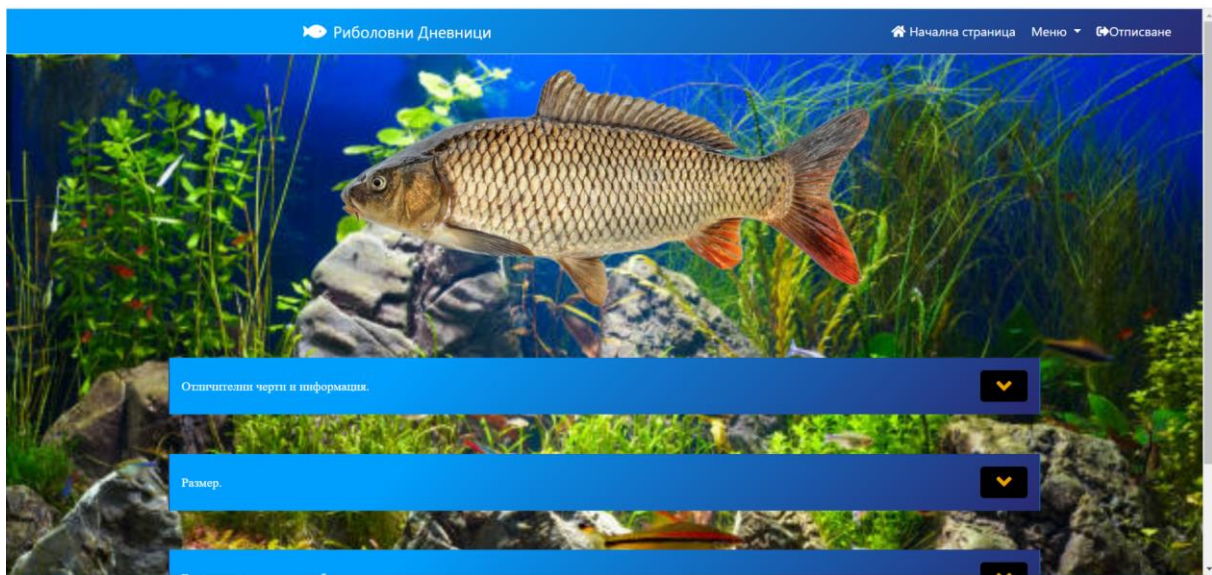


Фиг. 4.4 Списък със рибни видове

На тази страница могат да бъдат видени всички видове риба, които са достъпни в системата. Могат да бъдат преглеждани ръчно или да бъдат филтрирани по име чрез полето за търсене.

Имената на рибите служат като препращащ елемент , който води до по- подробна информация за вида риба.

На фиг. 4.5 може да се види страницата ,достъпна след кликуване върху името на определен вид риба, която съдържа по-детайлна информация за избраната риба .

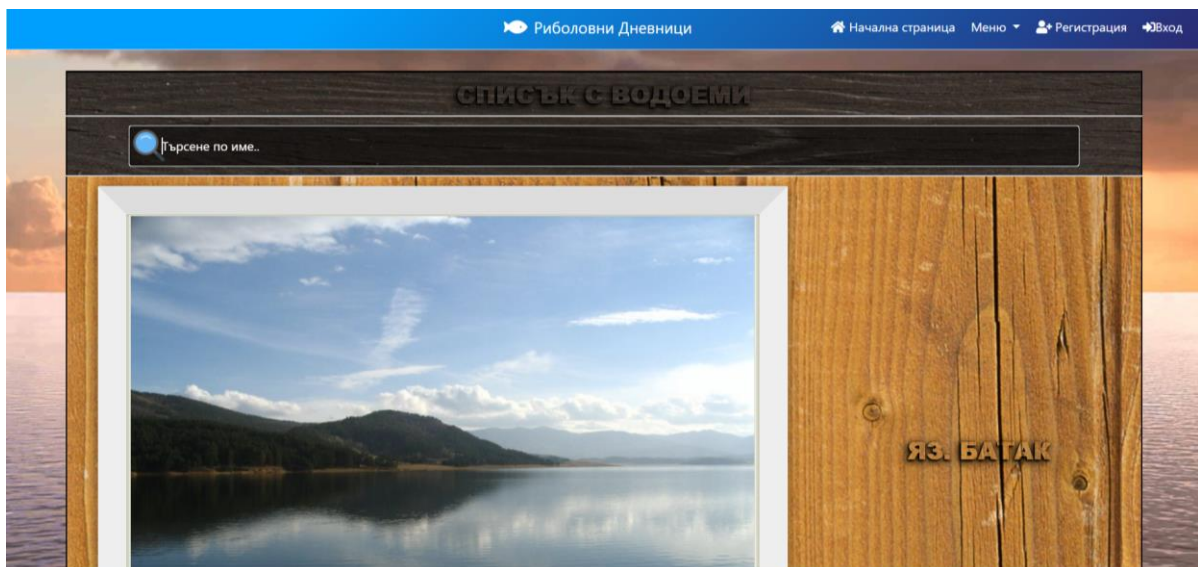


Фиг. 4.5 Детайлна информация за вида риба.

Страницата съдържа информация за вида риба като :

- Отличителни черти и информация – информация за отличителните черти на рибата, чрез които може да бъде разпозната и други интересни факти.
- Размер – ширина и тегло.
- Водоеми в които рибата може да бъде открита.
- Коментари – панел съдържащ всички коментари за вида риба , оставени от потребители, а също така и форма за оставяне на нов коментар която е активна за всички регистрирани потребители.

На фиг. 4.6 може да се види страницата съдържаща всички водоеми в системата.

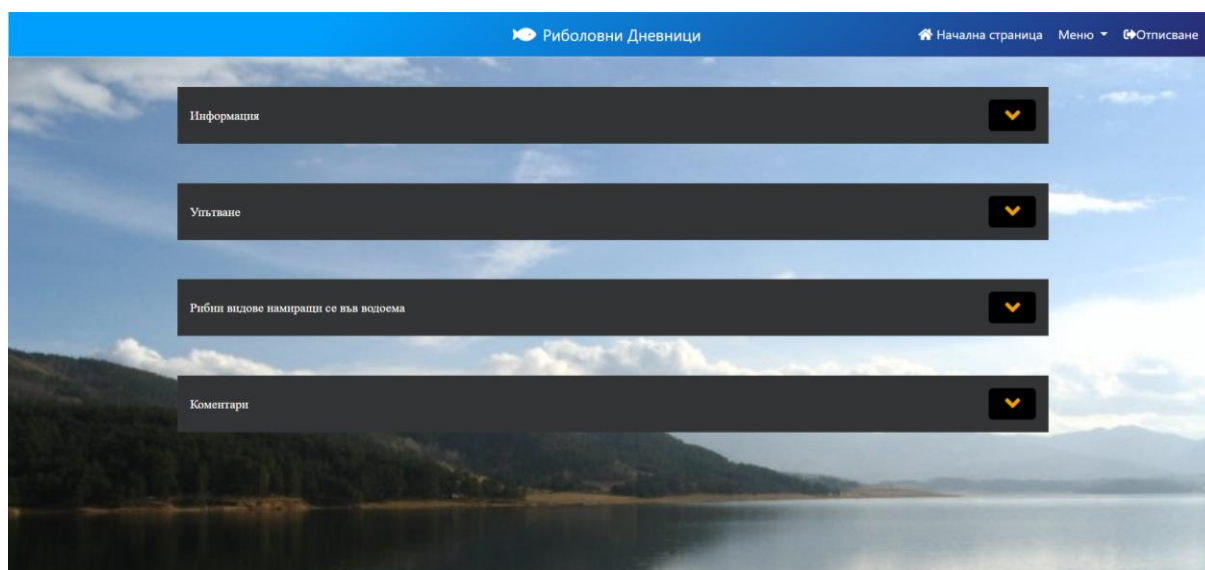


Фиг. 4.6 Списък със водоеми

На тази страница могат да бъдат видени всички водоеми, които са достъпни в системата. Могат да бъдат преглеждани ръчно или да бъдат филтрирани по име чрез полето за търсене.

Имената на водоемите служат като препращащ елемент , които води до по- подробна информация за тях.

На фиг. 4.7 може да се види страницата ,достъпна след кликване върху името на определен водоем, която съдържа по-детайлна информация за избрания водоем



Фиг. 4.7 Детайлна информация за избрания водоем.

Страницата съдържа следната информация за водоема:

- Историческа информация за водоема.
- Упътване – показва местоположението на водоема и съдържа препратка към Google maps за по-нататъшно упътване.
- Видовете риба, които могат да бъдат открити във водоема.
- Коментари – панел съдържащ всички коментари за водоема , оставени от потребители, а също така и форма за оставяне на нов коментар която е активна за всички регистрирани потребители.

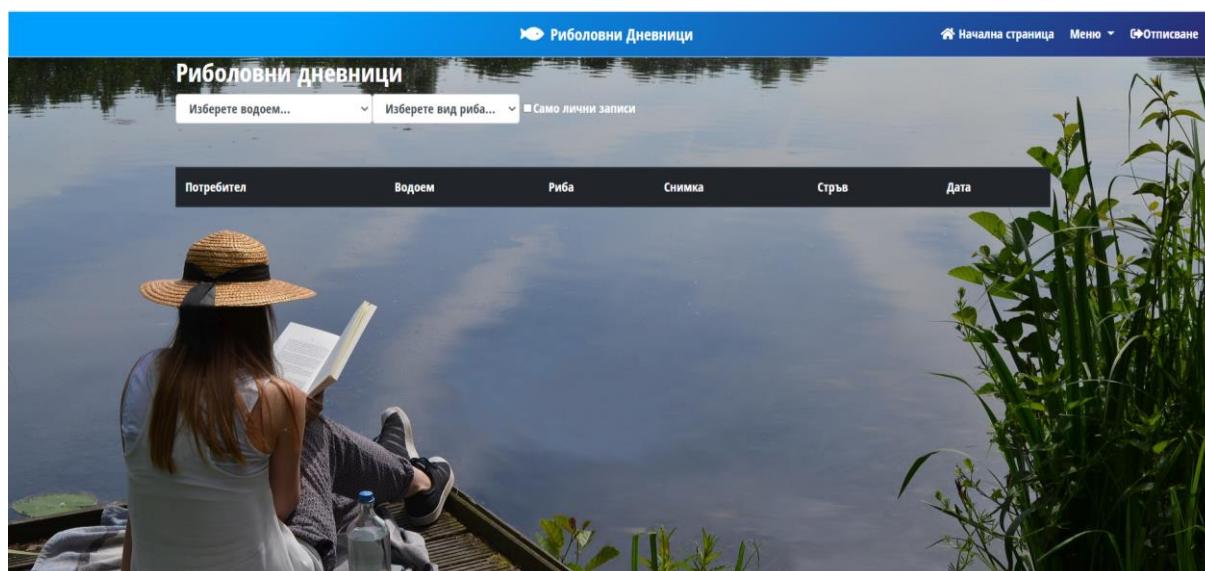
След успешен вход в системата с роля „Потребител“ в падащото меню са достъпни следните функционалности.

На фиг. 4.8 може да се види страницата съдържаща форма за създаване на запис на риболовен дневник.

Фиг. 4.8 Създаване на риболовен запис

Потребителя въвежда необходимата информация, след натискане на бутона „Изпращане“ информацията се изпраща за верификация. При коректно попълнена информация риболовният запис се вкарва в БД на системата. При непопълнено поле или некоректно попълнено такова, потребителя е върнат към страницата за въвеждане и получава текстово указание за грешката си.

На фиг. 4.9 може да се види страницата съдържаща списък със всички риболовни дневници, които могат да бъдат филтрирани по различни критерии.



Фиг. 4.9 Преглеждане на риболовните записи

Страницата служи за разглеждане на потребителските риболовни записи и тяхното филтриране по различни критерии. Записите могат да бъдат филтрирани по:

- Вид риба – Показва всички записи свързани с улавянето на точно определен вид риба.
- Водоем – Показва всички записи свързани с улов направен на точно определен водоем.
- Лични записи – Показва само записи направени от конкретния потребител използващ приложението.
- Комбинирано – Показва всички записи отговарящи на повече от един от гореописаните критерии.

На фиг. 4.10 може да се види страницата съдържаща форма за създаване на предложение за добавяне на нов вид риба в системата.

Фиг. 4.10 Предложение за нов вид риба

Потребителя въвежда необходимата базова информация за предложения вид риба ,като размер, име, и информация, а също така и допълнително разяснение защо мисли , че даденият вид риба трябва да бъде добавен. При коректно попълнени данни информацията се изпраща в системата и очаква одобрение/отказ от администратор. При некоректно попълване или пропуск на попълване на някое от полетата ,потребителя е върнат към страницата за въвеждане на предложението и получава информация за допуснатите грешки.

На фиг. 4.11 може да се види страницата съдържаща форма за създаване на предложение за добавяне на нов водоем в системата.

Потребителя въвежда необходимата базова информация за предложения водоем ,като местоположение, име, и информация, а също така и допълнително разяснение защо мисли , че даденият водоем трябва да бъде добавен.

Риболовни Дневници

Начална страница Меню Регистрация Вход

Въведете детайли за предложения от вас водоем.

Въведете името на водоема.

Въведете географската дължина на водоема.

Въведете географската ширина на водоема.

Въведете допълнителна информация за водоема.

Въведете допълнителна информация за предложението си.

Прикачете снимка на водоема ако разполагате с такава.

Избор на файл

Изпращане

Фиг. 4.11 Предложение за нов водоем.

При коректно попълнени данни информацията се изпраща в системата и очаква одобрение/отказ от администратор. При некоректно попълване или пропуск на попълване на някое от полетата, потребителя е върнат към страницата за въвеждане на предложението и получава информация за допуснатите грешки.

На фиг. 4.12 може да се види страницата съдържаща форма за създаване на предложение за добавяне на вид риба в даден водоем.

Риболовни Дневници

Начална страница Меню Отписване

Направете предложение за липсващите риби в някой от водоемите.

Изберете водоем...

Изберете вид риба...

Въведете допълнителна информация относно вашето предложение за да може да бъде преценена неговата валидност.

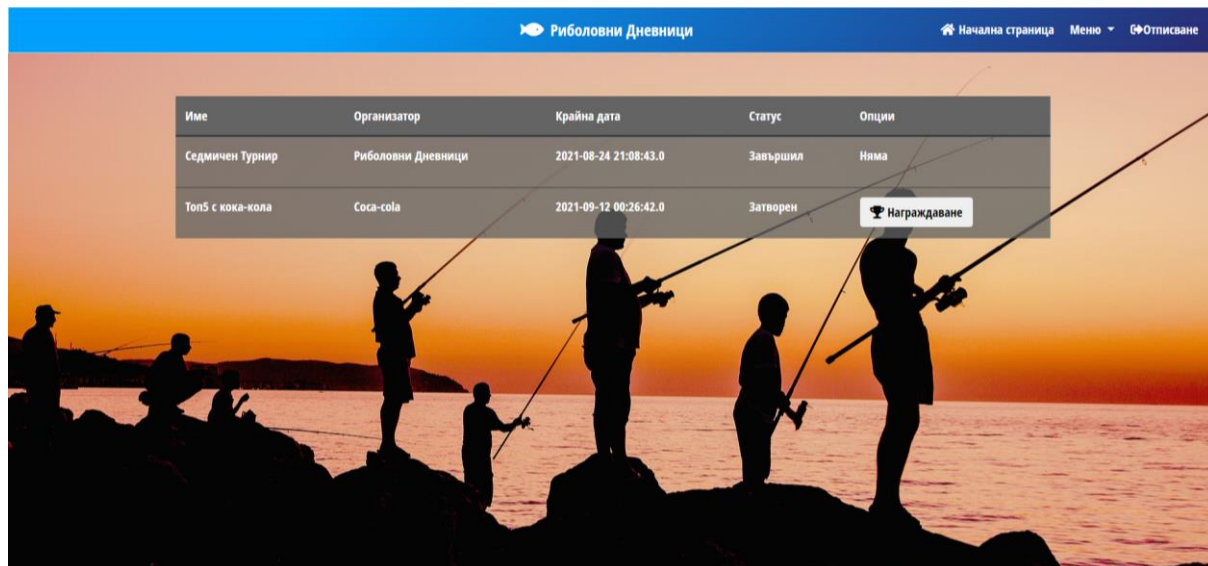
Изпращане

Фиг. 4.12 Предложение за риба във водоем.

Потребителя избира от падащите менюта вида риба които желае да добави и водоема в който желае да я добави, също така дава разяснение за причината поради която мисли ,че това предложение е валидно. След верификация предложението се изпраща за разглеждане от администратор. При некоректно попълване или пропуск при

попълване на някое от полетата потребителя е върнат към страницата за въвеждане и получава обратна връзка за допуснатата грешка.

На фиг. 4.13 може да се види страницата съдържаща списък с всички турнири протичащи в момента.



Фиг. 4.13 Списък с всички турнири

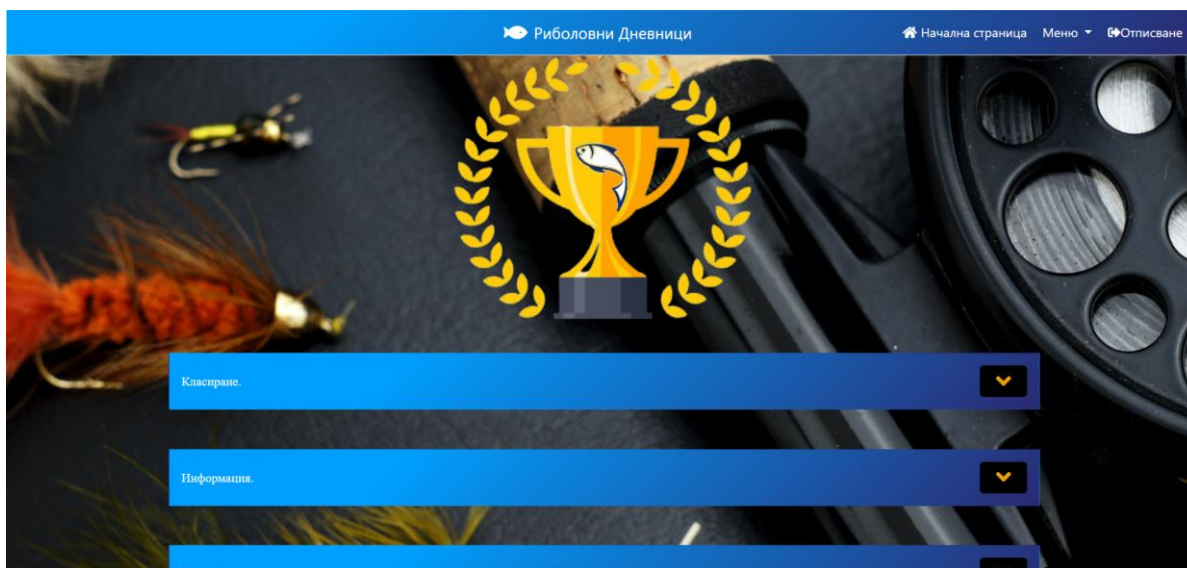
Страницата предоставя информация за всички провеждащи се в момента турнири, както и базова информация за самите турнири като:

- Организатор – човекът или организацията отговаряща за провеждане и спонсориране на турнира.
- Име на турнира.
- Статус – показва дали турнирът все още е активен или е приключил.
- Крайна дата за участие в турнира

Потребители с роля администратор могат също така да финализират награждаването за приключили турнири чрез опцията “Награждаване”.

Имената на турнирите служат като препращащ елемент , които води до по- подробна информация за тях.

На фиг. 4.14 може да се види страницата, която се достъпва чрез кликване върху името на някой от турнирите, съдържаща детайлна информация и класиране за избраният турнир.

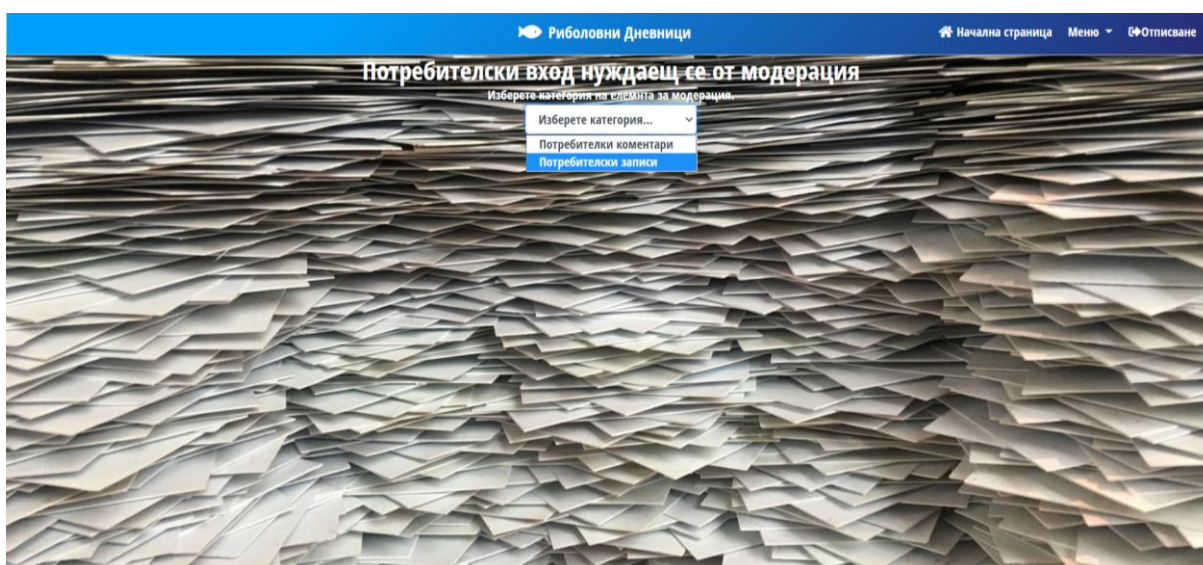


Фиг. 4.14 Детайлна информация за избрания турнир.

Страницата предоставя детайлна информация за избрания турнир като:

- Класиране – таблица с досегашното класиране на участниците в турнира включващо размера на техния улов и сегашната им позиция.
- Допълнителна информация за самия турнир и за организатора му.
- Награди – детайлна информация за наградите обвързани с турнира и правила за получаването им.

След успешно влизане във системата със роля „Модератор“ става достъпна следната допълнителна функционалност, която може да се види на фиг. 4.15 – страницата за модерация на потребителските записи и коментари.

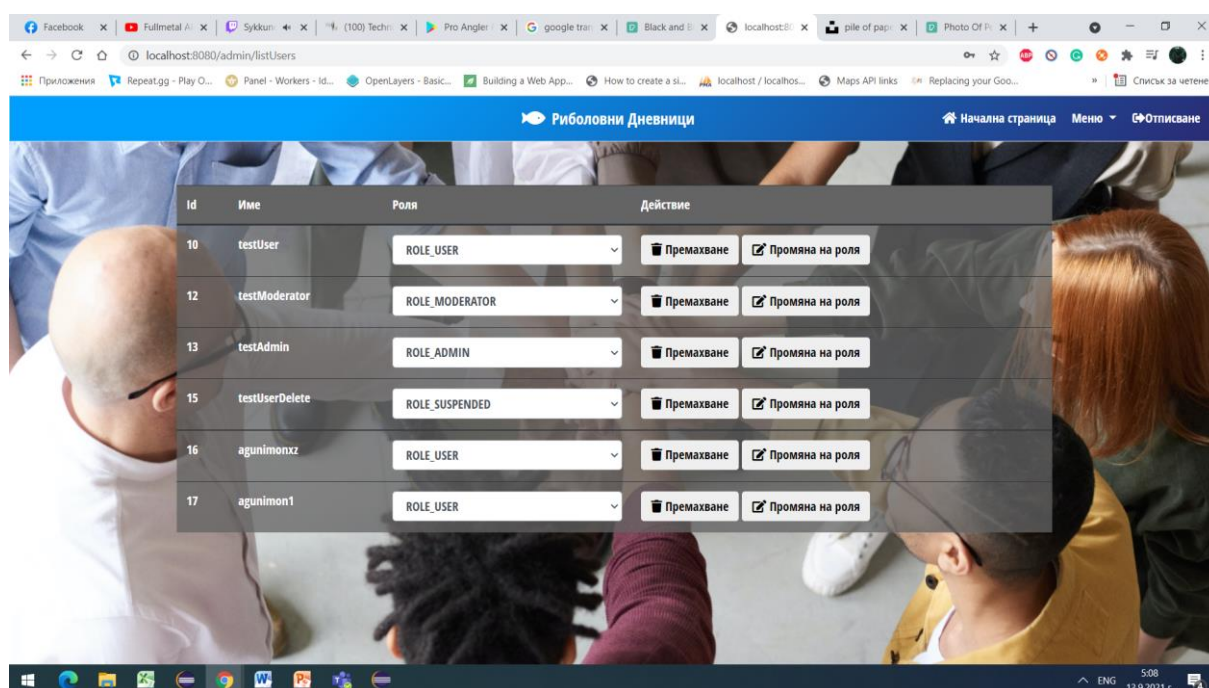


Фиг. 4.15 Модерация на потребителски вход

Страницата служи като средство за модерация на потребителския вход от потребители с роля модератор. Чрез падащо меню се избира вида на елемента на модерирание, като той може да бъде потребителски коментар или риболовен запис. Модераторът преценява валидността и коректността на елемента и може да „Приеме“ или „Отхвърли“ елемента според тази оценка.

Приетите елементи получават статус на „модерирани“ и стават валидни за преглеждане от всички потребители. Отхвърлените елементи се изтриват от системата.

След успешно влизане във системата със роля „Администратор“ стават достъпни следните допълнителни функционалности (На фиг. 4.16) в страницата за управление на потребителите.

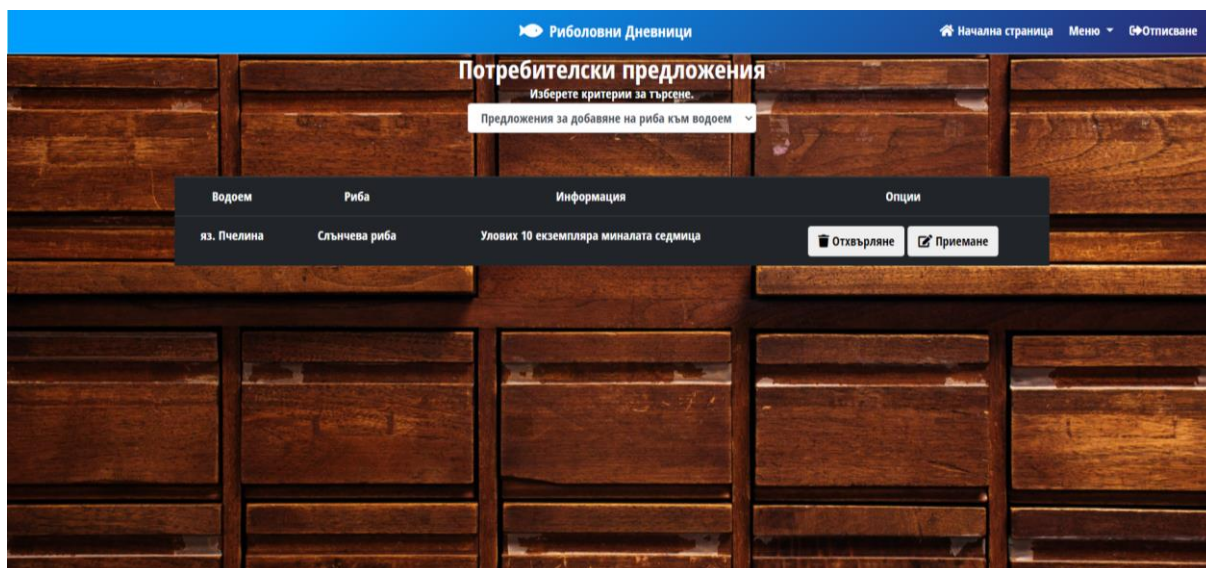


Фиг. 4.16 Управление на потребителите

Чрез тази страница администратора може да преглежда списъка с всички потребители и отредените им роли. Чрез падащото меню може да променя техните роли, като това служи за назначаване на нови администратори и модератори или за премахването на тези права. Чрез тази функционалност също така могат да бъде ограничаван достъпа до някои функционалности на обикновените потребители.

Чрез бутона „Премахване“ могат да бъдат премахвани потребители за неограничен период от време.

На фиг. 4.17 може да се види страницата за преглеждане и модерация на потребителските предложения, като видът на предложенията си избира чрез падащото меню.



Фиг. 4.17 Управление на потребителските предложения

Потребителят с роля администратор избира вида на предложенията които иска да прегледа, като те могат да бъдат:

- Предложения за нов вид риба.
- Предложения за нов водоем.
- Предложения за добавяне на риба във водоем.

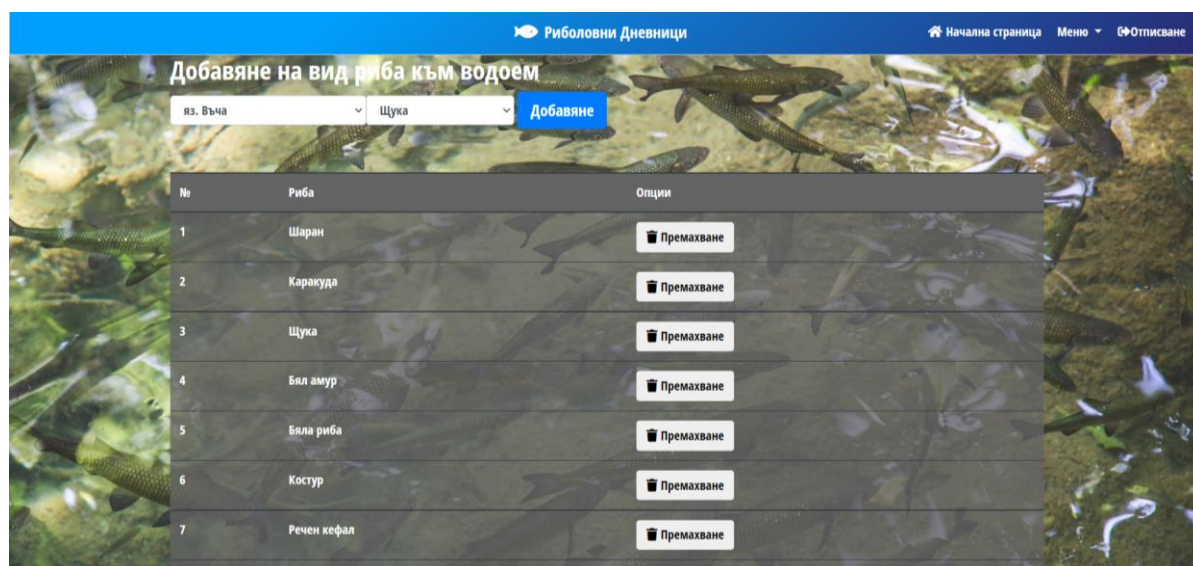
По преценка на администратора тези предложения биват приети или отхвърлени, като при приемане те стават част от БД за която се отнасят а при отхвърляне предложението се изтрива.

На фиг. 4.18 може да се види страницата съдържаща форма за въвеждане на нов турнир.

Фиг. 4.18 Създаване на турнир

Потребителя въвежда необходимата информация в означените за това полета. След натискане на бутона „Изпращане“ информацията се валидира. При коректно попълнени полета се създава нов запис за турнир в БД. При некоректно попълнено поле или пропуск при попълване на някое от полетата , потребителя се връща на страницата за попълване и получава насоки за своите грешки под формата на текст.

На фиг. 4.19 може да се види страницата за администратори за улеснено добавяне и премахване на риби от даден водоем.



Фиг. 4.19 Управление на съдържаните риби във водоем.

Страницата служи за улеснено добавяне на вид риба към даден водоем от потребители с роля администратор. Чрез предоставената форма администратора може да добавя , а също така и премахва риби от даден водоем , като се прескача процеса на предложение-одобрение. Тази функционалност се използва най-често за добавяне на голямо количество риби наведнъж, при добавянето на нов водоем в системата. Опцията за премахване служи за премахване на вид риба от водоем, поради грешка при въвеждането или промяна с течение на времето.

Заклучение

Разгледаното приложение ще допринесе за обогатяването на риболовната база на страната и ще ентусиазира всички потребителя да обиколят страната в търсене на желаня улов. Риболовните дневници ще помогнат на потребителите да запазват своите постижения и открития и ще се превърнат в тяхната гордост като рибари. Информацията придобита от сайта ще помогне на начинаещите рибари да започнат своята риболовна експедиция като ги упъти към най-подходящите места за тях и необходимото оборудване. Освен това информацията ще даде на по-напредналите и сезонни рибари нови техники и похвати, споделени от колеги рибари, с които те да станат по ефективни в начинанието си.

За по-нататъшното развитие на приложението могат да бъдат добавени следните функционалности:

- Онлайн магазин за стръв, за лесно пазаруване на необходимата стръв след проверка на информацията в уеб приложението. Па детайлно описание на самите примамки.
- Разработка на уникален маркер с логото на приложението, които да бъде използван при правенето на снимки за участие в турнири, чрез който да бъде допълнително валидирана уникалността и валидността на потребителските снимки.
- Добавяне на рейтингова система за потребителите за да може те да бъдат награждавани с по-висок рейтинг при допринасяне на полезна информация и рейтинга да бъде намаляван при нарушаване на правила или спам. Рейтинговата система ще подпомогне избирането на подходящи кандидати за модератори.

Използвани програмни продукти

- JDK 8
- Eclipse Java IDE
- MySQL Workbench
- PhpMyAdmin
- MySQL Connector/J
- Apache Tomcat
- CSS
- Javascript
- HTML
- BOOTSTRAP
- jQuery
- OpenLayers
- Ajax
- Spring boot
- Spring Security
- JPA

Използвана литература

1. W3Schools, <http://www.w3schools.com/>
2. Bootstrap , <http://getbootstrap.com/>
3. MySQL Database, <http://dev.mysql.com/>
4. Tutorialspoint.com, <https://www.tutorialspoint.com/index.htm>
5. Thymeleaf.org , <https://www.thymeleaf.org/>
6. Ken Arnold, James Gosling, David Holmes, THE Java™ Programming Language, <https://www.acs.ase.ro/Media/Default/documents/java/ClaudiuVinte/books/ArnoldGoslingHolmes06.pdf>
7. <https://www.thymeleaf.org/documentation.html>
8. International Conference on Science and Technology 2019, Data Structure Comparison Between MySql Relational Database and Firebase Database <https://iopscience.iop.org/article/10.1088/1742-6596/1569/3/032092/pdf>
9. Dragos-Paul Pop, AdamAltar, <https://www.sciencedirect.com/science/article/pii/S187770581400352X>
10. <https://openlayers.org/>